A Project Report

On

Medical Store Management System

By

TY I C Batch

Under the guidance of

Mr. Mahesh Kandekar



Department of Computer Engineering

Marathwada Mitra Mandal's College of Engineering

An Autonomous Institute

Permanently affiliated to

SAVITRIBAI PHULE PUNE UNIVERSITY

2026-2027

Marathwada Mitra Mandal's College of Engineering
Department of Computer Engineering,
Pune -51

CERTIFICATE

This is to certify that, of class T.Y. I C Batch, Computer have successfully completed their project work on "Medical Store Management System" at MARATHWADA MITRA MANDAL'S COLLEGE OF ENGINEERING in the partial fulfillment of the Graduate Degree course in T.Y. at the Department of Computer Engineering in the academic Year 2026-2027 Semester – as prescribed by the Savitribai Phule Pune University.

Mr. Mahesh Kandekar
Guide

Dr. S. S. Shiravale
Head of the Department
(Department of Computer Engineering)

Acknowledgement

I feel great pleasure in expressing my deepest sense of gratitude and sincere thanks to my guide Mr. Mahesh Kandekar for their valuable guidance during the Project work, without which it would have been a very difficult task. I have no words to express my sincere thanks for valuable guidance, extreme assistance and cooperation extended to all the **Staff Members** of my Department.

This acknowledgement would be incomplete without expressing my special thanks to **Dr. S. S. Shiravale**, Head of the Department (Computer Engineering) for their support during the work.

I would also like to extend my heartfelt gratitude to my **Principal, Dr. S. R. Sakhare** who provided a lot of valuable support, mostly being behind the veils of college bureaucracy.

Last but not least I would like to thank all the Teaching, Non- Teaching staff members of my Department, my parents and my colleagues who helped me directly or indirectly for completing this Project successfully.

Name of Students

Sumit Bhamare(B25CE2002)
Vedantika Mahadik(B25CE2003)
Shreya Shivade(B25CE2004)
Dnyaneshwari Thombal(B25CE2005)
Diksha Rupnawar(B25CE2008)
Shruti Dalvi(B25CE2009)
Tanvi Dongare(B25CE2010)
Omkar Kulkarni(B25CE2012)
Prarthana Kumbhar(B25CE2014)
Saniya Shaikh(B25CE2016)

INDEX

1	Introduction: Background, Problem statement, Objectives, Scope
2	Survey: Mapping of Fundamental Courses, Justification of Technology stack
3	Requirement analysis: Users, Functional and non-functional requirements, UML Designs
4	KPI framework
5	System design and Architecture
6	Front End Design & Development
7	Back End Design & Development
8	Logic Layer Design & Development
9	Testing and Integration
10	Deployment
11	Results and KPI achievement
12	Team contribution and risks identifications
13	Weekly Templates (Team Wise)
14	Implementation Screen Shots
15	Conclusion and future scope
	References

1. INTRODUCTION

1.1 Background

Medical stores manage various activities, including medicine sales, inventory management, supplier records, and expiry tracking. Handling these activities manually can be time-consuming and may lead to billing errors, incorrect stock records, and expired medicines remaining in inventory.

The **Medical Store Management System (MSMS)** is a web-based application designed to simplify and automate these operations. It provides medicine billing, batch-wise stock tracking, expiry alerts, supplier management, prescription records, and report generation. The system also uses the **First Expiry, First Out (FEFO)** method to prioritize medicines that expire earlier, helping improve inventory control and operational efficiency.

1.2 Problem Statement

Traditional medical store management methods often involve manual records or disconnected systems, making it difficult to maintain accurate stock information and manage medicine expiry dates. Billing errors, delays in customer service, improper return handling, and difficulty tracking suppliers and purchases can affect daily operations.

Therefore, a centralized system is required to automate billing, maintain accurate inventory records, track medicine batches and expiry dates, and generate useful reports while ensuring reliable and secure data management.

1.3 Objectives

The main objectives of the Medical Store Management System are:

1. To automate medicine billing and invoice generation.
2. To maintain accurate batch-wise medicine stock records.
3. To track expiry dates and generate low-stock and expiry alerts.
4. To implement FEFO-based stock deduction.
5. To manage suppliers, purchases, and purchase orders.
6. To maintain prescription records and link them to relevant sales.
7. To process medicine returns and update stock accordingly.
8. To generate sales, inventory, and expiry reports.
9. To provide role-based access for different users.
10. To improve the accuracy, security, and efficiency of medical store operations.

1.4 Scope

The Medical Store Management System covers the major operations of a medical store through a centralized web-based platform. Its scope includes the following modules:

1. **Authentication and Access Control:** Provides secure login and role-based access for users.
2. **Medicine and Category Management:** Supports adding, updating, and managing medicine details and reorder levels.

3. **Supplier and Purchase Management:** Maintains supplier information, purchase orders, and batch details.
4. **Billing and Point of Sale:** Enables medicine search, cart management, customer billing, and invoice generation.
5. **Stock and Expiry Tracking:** Tracks batch-wise inventory, applies FEFO, and generates low-stock and expiry alerts.
6. **Prescription Management:** Stores prescription records and links them to applicable sales.
7. **Returns Processing:** Records medicine returns, updates stock, and stores return reasons.
8. **Reports and Dashboard:** Generates sales, stock, expiry, and low-stock reports, along with PDF exports.

The system is intended for pharmacists, cashiers, store managers, and administrators. It aims to reduce manual work, improve inventory accuracy, and support efficient day-to-day medical store management.

2. SURVEY

2.1 Mapping of Fundamental Courses

The Medical Store Management System integrates concepts from various fundamental courses studied in the engineering curriculum.

Table 2.1: Mapping of Fundamental Course

Sr. No.	Fundamental Course	Application in the Project
1	Object-Oriented Programming (OOP)	Organizes the system using classes and objects for medicines, users, suppliers, and bills.
2	Database Management Systems (DBMS)	Stores and manages medicine, stock, supplier, prescription, and billing records.
3	Data Structures	Uses lists, queues, and other suitable structures for managing billing carts and customer queues.
4	Software Engineering	Supports requirement analysis, system design, testing, and maintenance.
5	Web Technology	Enables users to access the system through a web-based interface.
6	Principles of Programming Languages	Applies programming concepts such as data types, control structures, functions, and modularity.

2.2 Justification of Technology Stack

The technology stack is selected to make the system user-friendly, reliable, maintainable, and suitable for managing medical store operations.

Table 2.2: Justification of Technology Stack

Technology	Purpose	Justification
HTML	Web page structure	Organizes forms, tables, and page content.
CSS	User interface styling	Provides a clear and consistent layout.
JavaScript	Client-side functionality	Supports interactive billing forms, validation, and dynamic updates.
Backend Technology	Business logic and processing	Handles authentication, billing, stock updates, and return processing.
Relational Database (e.g., MySQL)	Data storage	Maintains structured and related records with data consistency.
PDF Generation Library	Report and invoice export	Allows invoices and reports to be generated in PDF format.

The selected technologies support modular development and efficient data management. The frontend provides an accessible interface, while the backend and database handle business rules and maintain consistent records. The final technology choices should match the tools used in the actual implementation.

3. Requirement Analysis: Users, Functional and Non-Functional Requirements, UML Designs

3.1 Users of the System

The Medical Store Management System is designed for the following users:

1. Pharmacist/Cashier: Searches medicines, processes customer bills, generates invoices, manages prescription-linked sales, and handles permitted medicine returns.

2. Administrator: Manages user accounts, assigns roles and permissions, and oversees the overall functioning of the system.

3.2 Functional Requirements

Functional requirements describe the operations that the system must perform.

1. **Authentication and Access Control:** The system shall provide login functionality and restrict access according to user roles.

2. **Medicine and Category Management:** The system shall allow authorized users to add, update, search, and deactivate medicine records.
3. **Supplier and Purchase Management:** The system shall maintain supplier details, create purchase orders, and record newly received medicine batches.
4. **Billing and Point of Sale:** The system shall support medicine searching, cart management, bill calculation, and invoice generation.
5. **Stock and Expiry Tracking:** The system shall maintain batch-wise stock quantities, monitor expiry dates, and generate low-stock and expiry alerts.
6. **Prescription Management:** The system shall allow prescription capture or upload and link prescriptions to applicable sales.
7. **Returns Processing:** The system shall record medicine returns, store return reasons, and perform appropriate stock adjustments.
8. **Report and Dashboard Generation:** The system shall generate sales, stock, expiry, and low-stock reports and support PDF export.
9. **Transaction Management:** The system shall process invoice generation, stock deduction, and ledger updates as one atomic transaction.
10. **Customer Queue Management:** The system shall maintain an ordered billing queue to support fair customer processing.

3.3 Non-Functional Requirements

Non-functional requirements describe the quality, performance, and reliability expected from the system.

1. **Security:** Only authorized users shall access restricted information and operations.
2. **Performance:** Medicine searches, billing operations, and report generation should respond within acceptable time limits.
3. **Reliability:** The system shall handle transaction failures without leaving incomplete billing or stock updates.
4. **Usability:** The interface shall be simple, clear, and easy to navigate.
5. **Data Integrity:** The system shall maintain accurate relationships between medicines, batches, bills, prescriptions, and returns.
6. **Maintainability:** The application shall use a modular structure to simplify debugging and future modifications.
7. **Scalability:** The system should support increasing numbers of medicines, users, and transactions.
8. **Availability:** The system should be accessible to authorized users during normal medical store operating hours.

3.4 UML Designs

3.4.1 Use Case Diagram

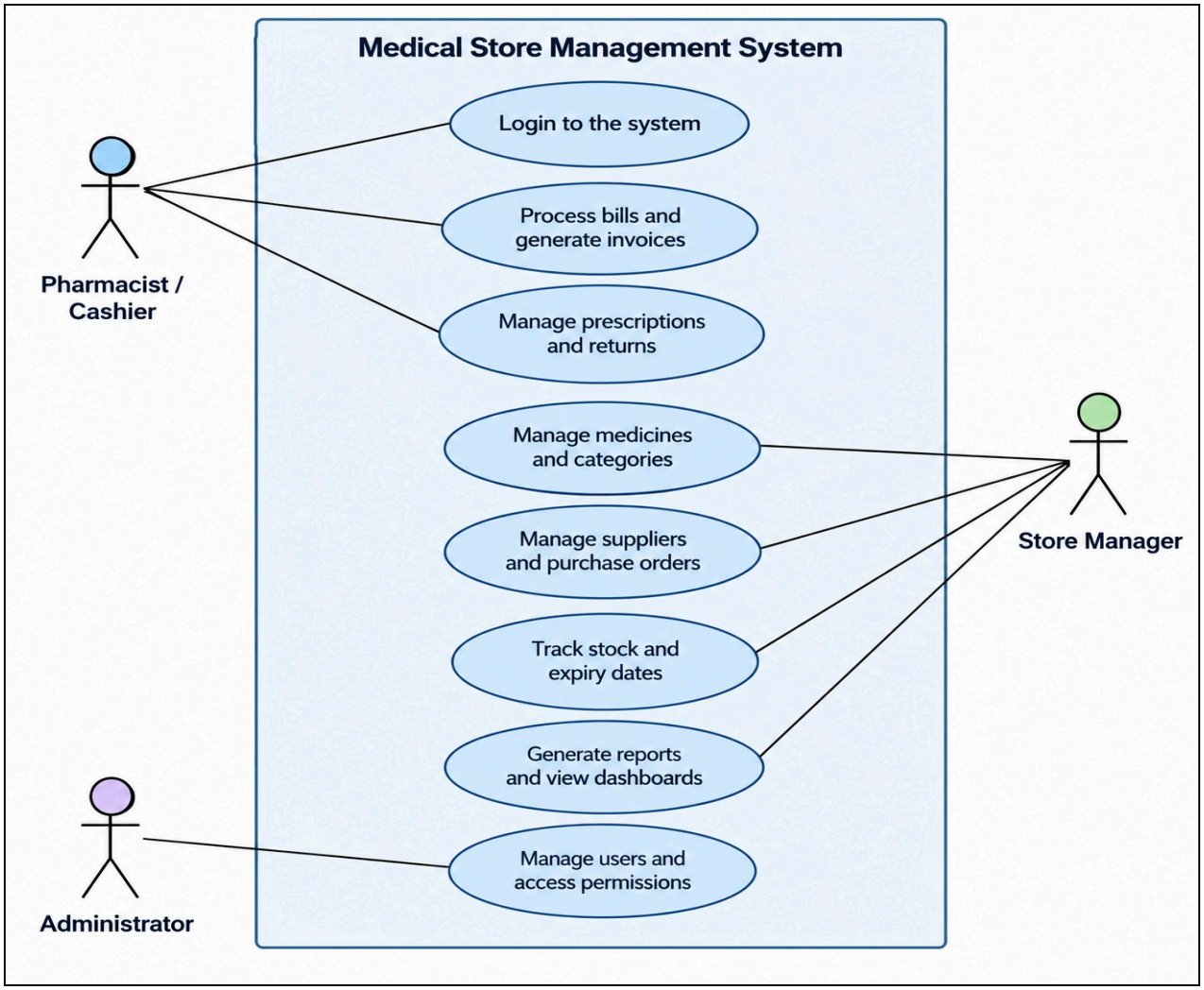


Fig 3.4.1 Use Case Diagram

3.4.2 Class Diagram

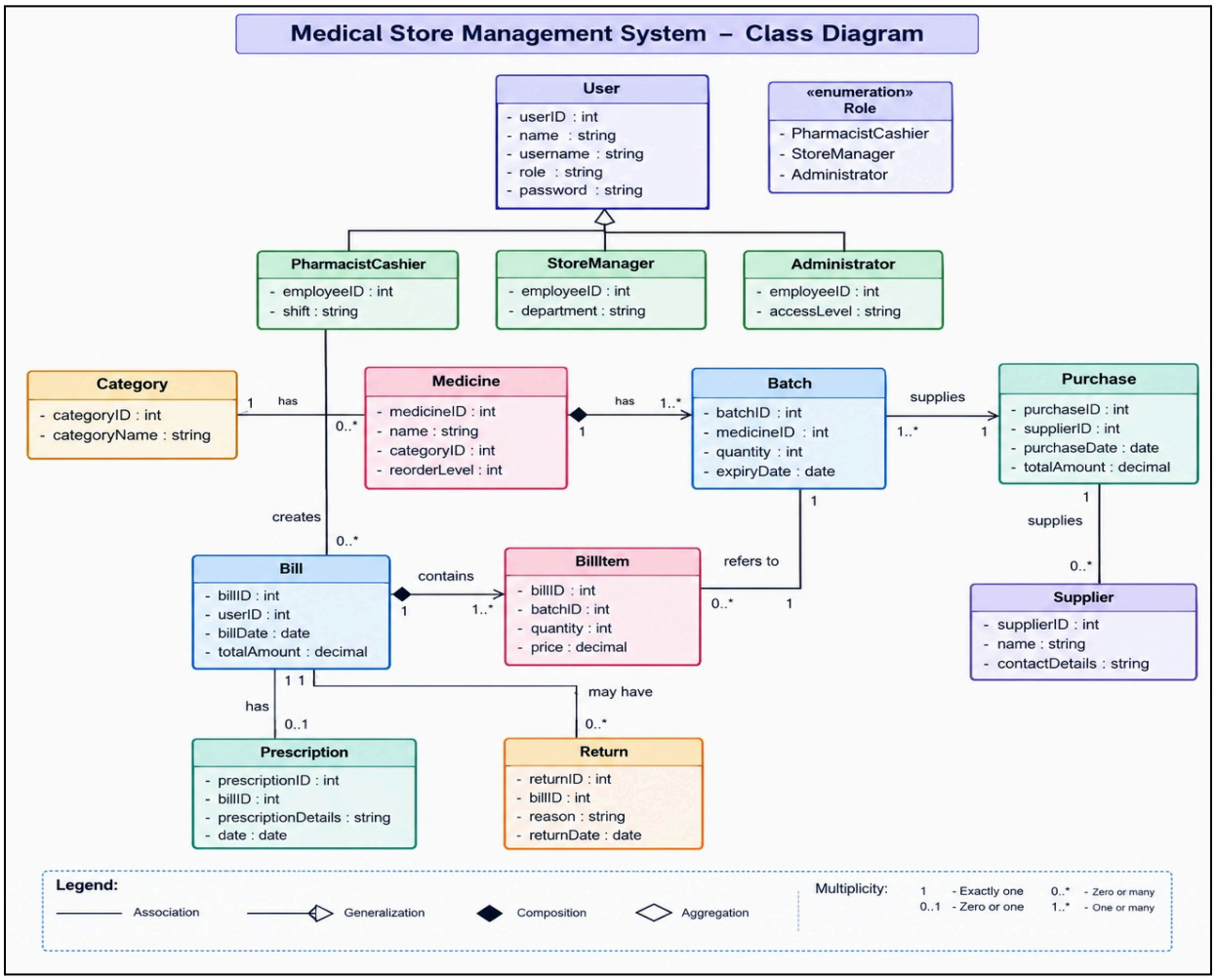


Fig 3.4.2 Class Diagram

3.4.3 Activity Diagram

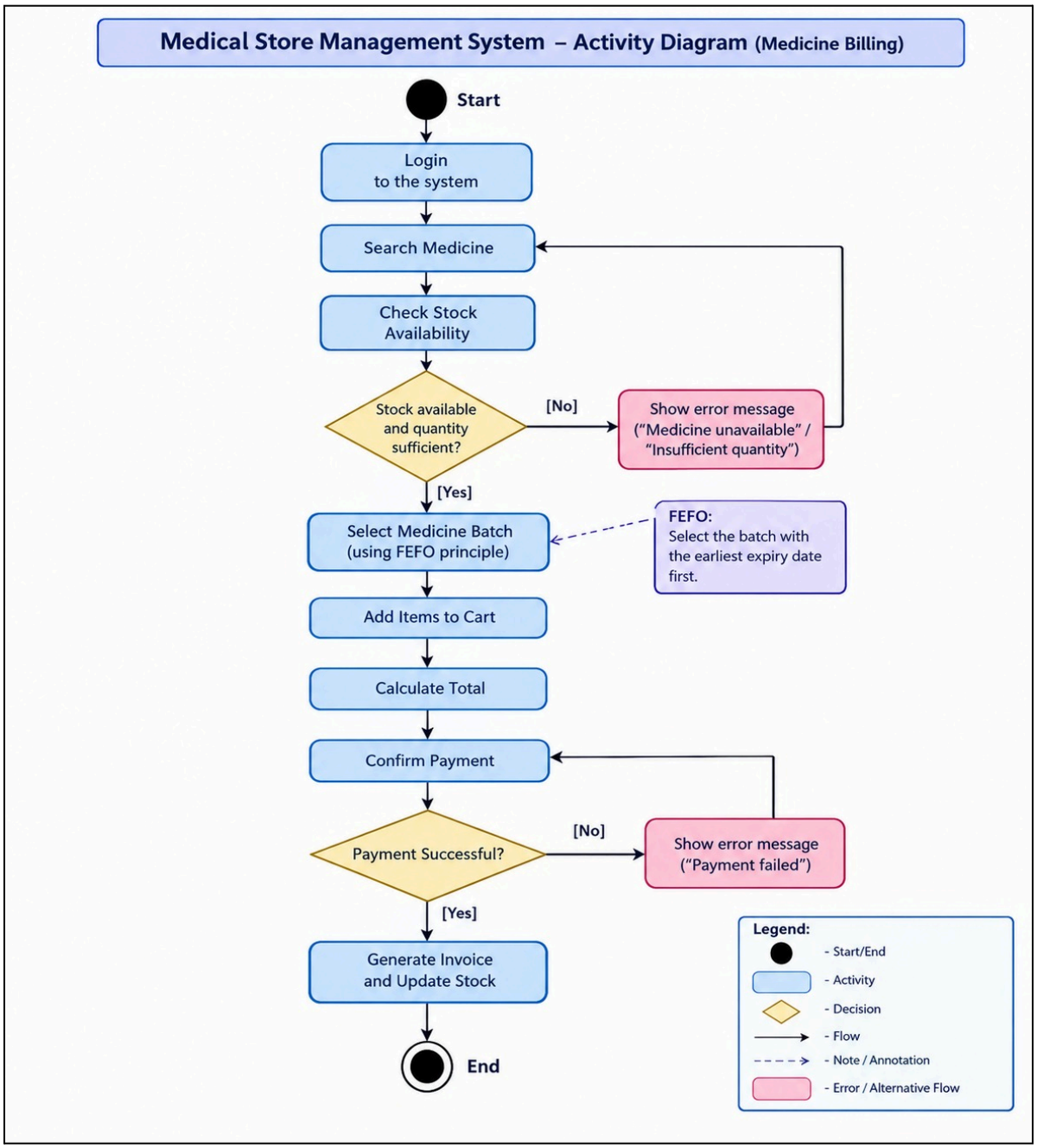


Fig. 3.4.3. Activity Diagram

3.4.4 Sequence Diagram

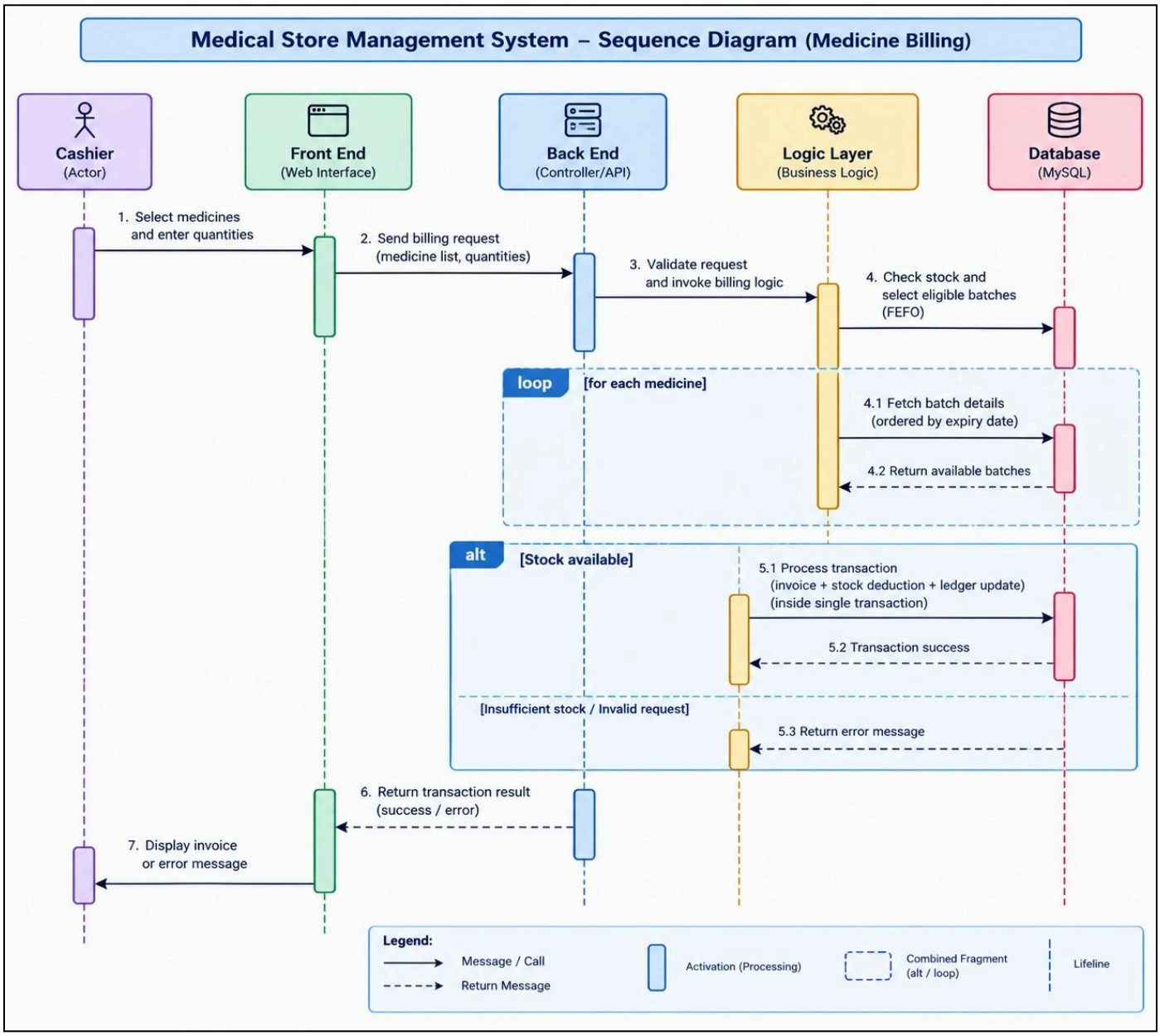


Fig. 3.4.4. Sequence Diagram

4. KPI FRAMEWORK

4.1 Key Performance Indicators

Table 4.1: Key Performance Indicators

Sr. No.	KPI	Purpose	Measurement
1	Billing Accuracy	Measures the correctness of generated bills.	Percentage of error-free bills
2	Billing Time	Evaluates the time required to complete a billing transaction.	Average time per bill
3	Inventory Accuracy	Checks whether recorded stock matches actual stock.	Percentage of matching stock records
4	Expiry Management	Measures the effectiveness of expiry tracking.	Number of expired medicines detected and tracked
5	Low-Stock Alerts	Evaluates timely identification of medicines requiring replenishment.	Number of low-stock alerts generated
6	FEFO Compliance	Checks whether batches with earlier expiry dates are prioritized.	Percentage of eligible sales following FEFO
7	Transaction Consistency	Ensures billing and stock updates remain consistent.	Number of failed or inconsistent transactions
8	Report Generation Time	Measures how quickly the system generates reports.	Average time required to generate a report
9	System Usability	Evaluates how easily users can operate the system.	User feedback and task completion rate
10	Return Processing	Measures the accuracy of medicine return and stock adjustment operations.	Percentage of correctly processed returns

5. SYSTEM DESIGN AND ARCHITECTURE

5.1 System Design

The Medical Store Management System (MSMS) is designed as a web-based application that automates the major operations of a medical store. The system provides a centralized platform for managing medicines, suppliers, purchases, billing, stock, prescriptions, returns, and reports. It follows a modular design in which each module performs a specific function while communicating with other modules through defined interfaces.

The system consists of three major layers: the Presentation Layer, Application Layer, and Data Layer. The Presentation Layer provides the user interface, the Application Layer processes requests and applies business rules, and the Data Layer stores and retrieves information from the database.

The system supports three main user roles: Pharmacist/Cashier, Store Manager, and Administrator. Each role is provided access to the functions required for its responsibilities. During billing, the system validates stock availability, selects eligible batches using the First Expiry, First Out (FEFO) principle, calculates the bill, and records the transaction. The invoice, stock deduction, and ledger entry are designed to be processed as one atomic transaction to maintain data consistency.

5.2 System Architecture

The system follows a three-tier architecture, which separates the user interface, application logic, and database operations.

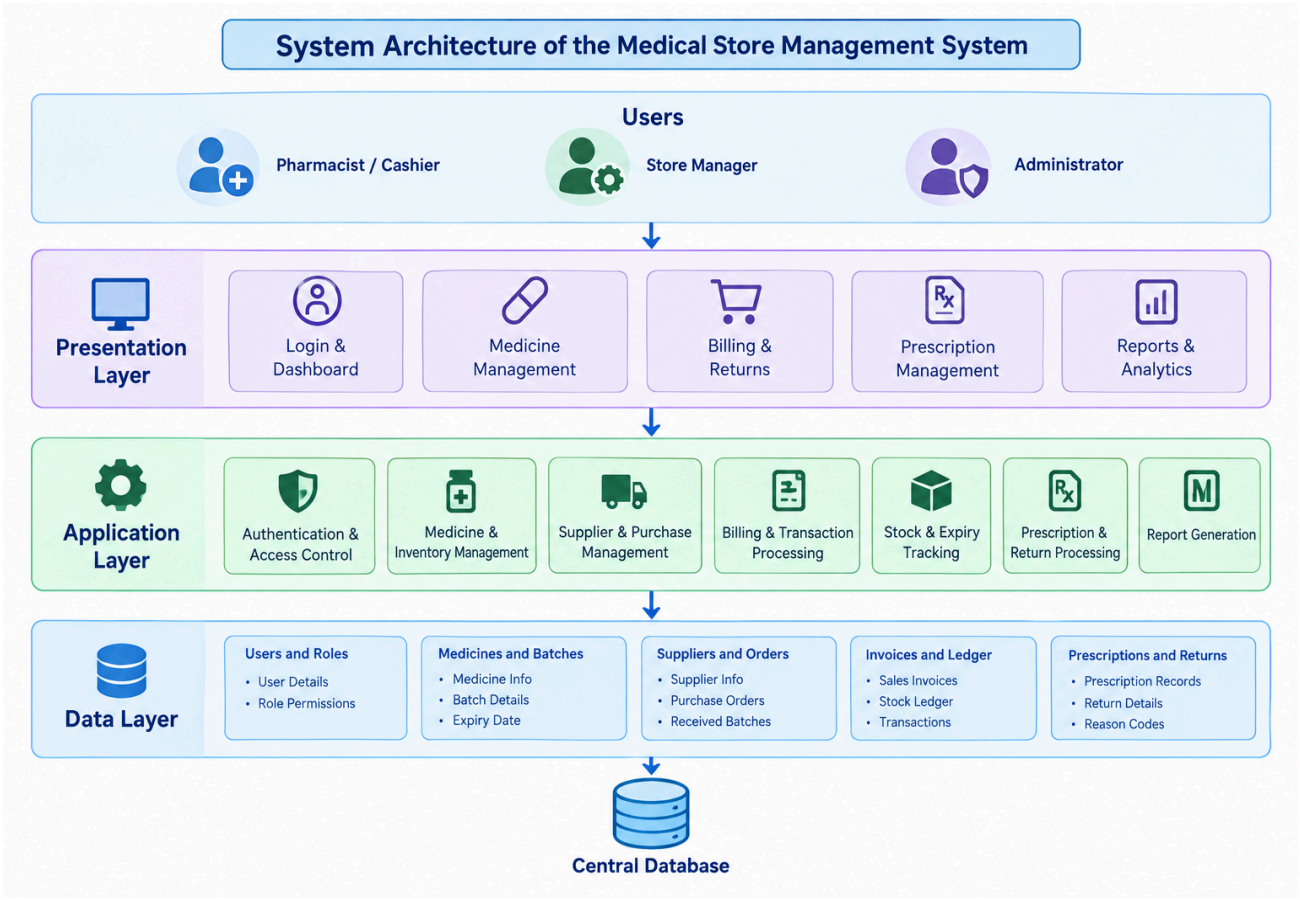


Fig. 5.2 System Architecture

Presentation Layer: Provides screens and forms for users to interact with the system. It supports medicine searches, billing, prescription management, return processing, and report viewing.

Application Layer: Handles request validation, business rules, authentication, stock calculations, billing, returns, and report generation. It coordinates the operations required to complete each user request.

Data Layer: Manages communication with the database. It stores and retrieves medicine, batch, supplier, purchase, billing, prescription, and return information while maintaining relationships and data integrity.

5.3 Module Architecture

The system is divided into eight functional modules:

1. Authentication and Access Control: Handles login, sessions or tokens, and role-based permissions.
2. Medicine and Category Management: Maintains medicine details, categories, and reorder levels.
3. Supplier and Purchase Management: Stores supplier information, purchase orders, and incoming batch details.
4. Billing / Point-of-Sale: Supports customer queues, medicine selection, cart management, and invoice generation.
5. Stock and Expiry Tracking: Maintains batch quantities, expiry dates, FEFO stock selection, and stock alerts.
6. Prescription Management: Captures or uploads prescriptions and links them to relevant sales.
7. Returns Processing: Records return details, reason codes, and appropriate stock reversals.
8. Report and Dashboard Generation: Produces sales, inventory, expiry, and low-stock reports, including PDF exports.

Each module is responsible for its own operations but shares relevant data with other modules. For example, billing interacts with inventory to check available stock, prescription management when a prescription is required, and reporting to maintain sales records.

5.4 Database Design

The database stores the information required for daily medical store operations. A relational database such as MySQL can be used to maintain structured records and relationships.
The main entities include:

- Users: Stores user details, login information, and role references.
- Roles: Defines the permissions assigned to different user types.
- Categories: Stores medicine category information.
- Medicines: Stores medicine names, descriptions, categories, and reorder levels.
- Batches: Stores batch numbers, quantities, expiry dates, and medicine references.
- Suppliers: Maintains supplier details.
- Purchase Orders: Stores purchase order information and associated items.
- Prescriptions: Stores prescription references and related information.
- Invoices: Stores billing details, dates, and totals.
- Invoice Items: Stores medicines, quantities, prices, and batch references for each invoice.
- Returns: Stores return dates, reasons, and transaction references.

- Return Items: Stores the medicines and quantities associated with each return.
- Stock Ledger: Records stock movements caused by purchases, sales, and returns.

The database uses primary keys to identify records and foreign keys to establish relationships. Appropriate constraints and transactions help prevent invalid references and inconsistent updates.

5.5 Billing and Transaction Workflow

The billing process is an important part of the system because it connects medicine selection, stock management, invoicing, and transaction records.

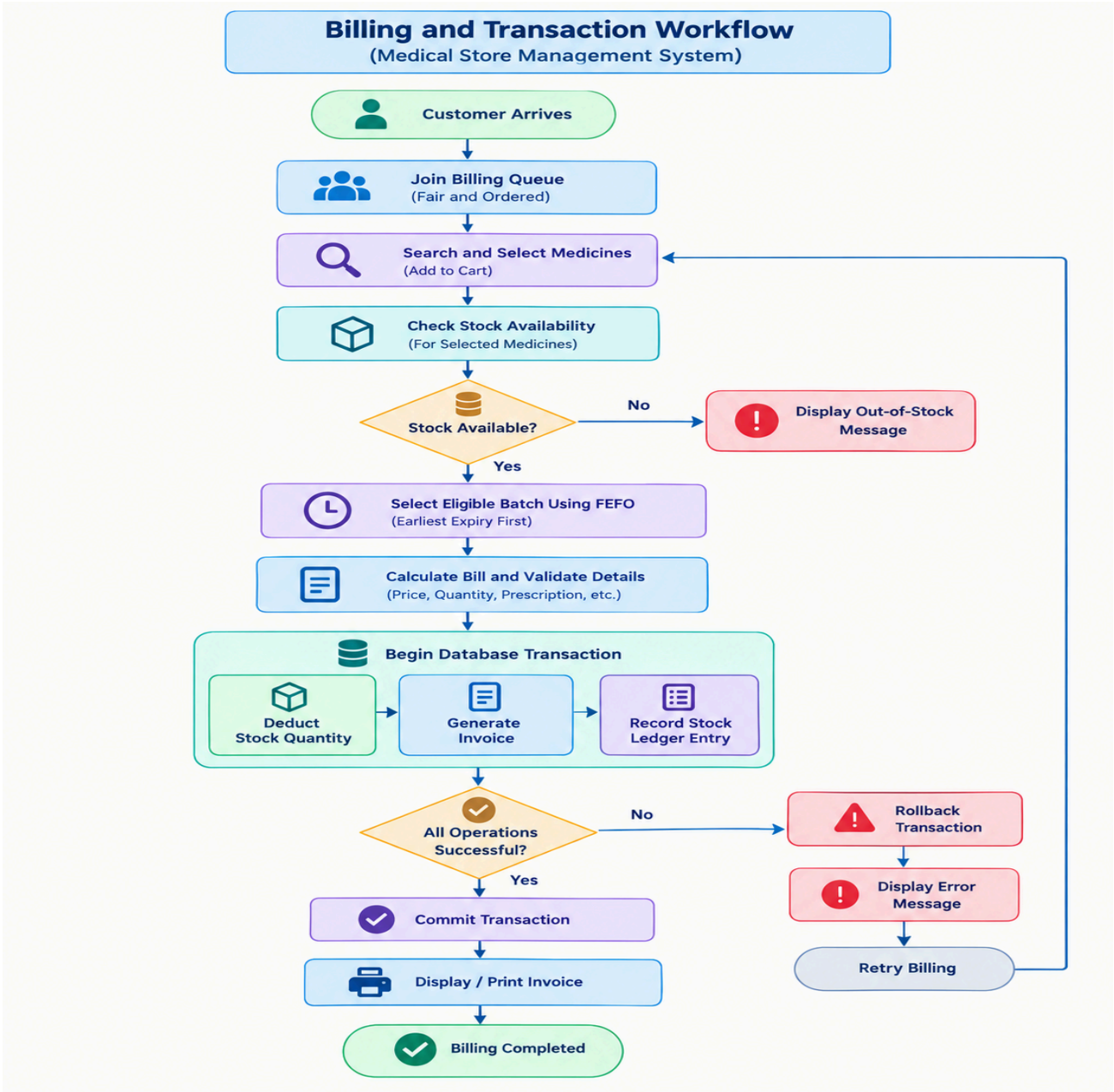


Fig. 5.5 Billing and Transaction Workflow

If any required database operation fails, the transaction should be rolled back so that partial changes are not saved. The system should also prevent stock quantities from becoming negative and ensure that expired batches are not selected for sale.

5.6 Security and Reliability

The system uses authentication and role-based access control to restrict operations according to user responsibilities. Input validation is applied to important forms and requests to reduce invalid data entry. Database constraints and transaction management help maintain data consistency during billing and return processing.

Regular verification of stock records, clear error messages, and appropriate handling of failed requests improve reliability. Testing should cover individual functions, communication between modules, complete workflows, and security-related scenarios.

5.7 Summary

The proposed architecture separates the user interface, application logic, and database operations to support modular development and maintenance. The eight modules work together to automate medical store activities, maintain batch-wise inventory, process bills and returns consistently, manage prescription records, and generate useful reports. This design provides a structured foundation for developing a reliable and efficient Medical Store Management System.

6. Front End Design & Development

The front end of the Medical Store Management System (MSMS) is a server-rendered, multi-page web interface. Each screen is a PHP script that produces HTML, styled with Bootstrap 5 and a custom stylesheet, and enhanced with jQuery for interactive behaviour such as the billing cart. No single-page-application framework (for example React, Angular or Vue) is used. The application is branded “PharmaCare” in the interface (APP_NAME in config.php).

6.1 Tools, Frameworks, and Libraries Used

The libraries below were identified from the <link> and <script> tags in includes/header.php, includes/footer.php, pages/login.php and pages/reports.php. All third-party front-end libraries are loaded from public CDNs, so an Internet connection is required when the application is used.

Table 6.1: Front-End Technologies

Component	Technology	Version	Purpose
Markup	HTML5 (generated by PHP templates)	HTML5	Page structure, forms and tables
Styling framework	Bootstrap (jsDelivr CDN)	5.3.0	Grid layout, forms, tables, modals, alerts, dropdowns, badges
Custom styling	assets/css/style.css	–	Sidebar, login screen, colour variables and responsive rules (187 lines)
Scripting	JavaScript (ES6) with jQuery (code.jquery.com)	jQuery 3.6.0	Cart handling, AJAX calls, calculation of totals, dynamic UI
Site-wide script	assets/js/main.js	–	Auto-hides alerts after 5 seconds; initialises Bootstrap tooltips
Icons	Font Awesome (cdnjs)	6.4.0	Sidebar and button icons
Typography	Google Fonts – Nunito	Weights 400–800	Consistent, readable typeface
Charts	Chart.js (jsDelivr CDN)	Not pinned (latest)	Line and doughnut charts on the Analytics & Reports page
Avatar image	ui-avatars.com (external image service)	–	User initials avatar in the top navigation bar

The version numbers of Bootstrap, jQuery and Font Awesome are taken directly from the CDN URLs in the source code. Chart.js is referenced without a version number, so its exact version cannot be stated.

6.2 UI/UX Design

The interface follows a conventional administrative-dashboard layout that is shared by all authenticated screens through three include files:

•**includes/header.php** – opens the HTML document, loads the libraries, enforces login (requireLogin()) and renders the top navigation bar with the signed-in user's name and a Logout option.

•**includes/sidebar.php** – renders the left navigation menu grouped under Dashboard, Inventory, Transactions, People and Management headings. The active page is highlighted.

• **includes/footer.php** – closes the layout and loads the Bootstrap bundle, main.js and the sidebar-toggle script.

Design decisions evident in the implementation are:

•**Task-based navigation:** menu items are grouped by purpose (inventory, transactions, people, management), which keeps the number of clicks to common tasks low.

•**Status colour coding:** helper functions in includes/functions.php render Bootstrap badges – Out of Stock (red), Low Stock (yellow) and In Stock (green) for quantity, and Expired (red), Expiring Soon (yellow, within 30 days) and Valid (blue) for expiry dates.

•**Immediate feedback:** success and error messages are shown as dismissible Bootstrap alerts (flash messages) that fade out automatically.

• **Searchable lists:** the medicines, suppliers, purchases and sales-history pages provide filters and search boxes to reduce manual scanning.

• **Role-aware menu:** Suppliers, Users & Staff, Purchase Orders and Analytics & Reports are displayed only to users whose role is admin.

Wireframes and style guide. The project does not contain separate wireframe files or a style-guide document. The visual style is defined by the Bootstrap defaults together with the colour variables and component rules in assets/css/style.css. **Accessibility.** The interface uses <label> elements for form fields, role="alert" and aria-label attributes on alert-close buttons, aria-expanded on the user dropdown, and a viewport meta tag. A formal accessibility audit (for example colour-contrast or screen-reader testing) has not been carried out.

6.3 Screens and Component Structure

Table 6.2 lists the screens implemented in the pages/ directory. Figures 6.1 to 6.4 show the login, dashboard, billing and inventory screens as captured from the application.

Table 6.2: Screens of the Application

Screen	File	Description	Access
Login	pages/login.php	Username/password form with CSRF token; redirects to the dashboard on success	Public
Dashboard	pages/dashboard.php	Summary cards (total medicines, today's revenue, low-stock count, expiry alerts, patients, profit), recent sales and tabbed low-stock /	Logged-in users

		expiring-soon lists	
Medicines	pages/medicines.php, pages/medicine_form.php	List with search and filters (category, stock status, expiry status); add, edit and delete	Logged-in users
Categories	pages/categories.php	Category list with medicine count; add, edit and delete (delete blocked if medicines exist)	Logged-in users
POS / Billing	pages/pos.php	Medicine search, cart, patient selection, discount and sale completion	Logged-in users
Invoice	pages/invoice.php, pages/generate_invoice.php	Printable HTML invoice and FPDF-generated PDF invoice	Logged-in users (see §7.5)
Sales History	pages/sales_history.php	Filterable list of sales with CSV export	Logged-in users
Patients	pages/patients.php, pages/prescriptions.php	Patient records and per-patient prescription records linked optionally to a sale	Logged-in users
Suppliers	pages/suppliers.php	Supplier directory; add, edit, delete (delete blocked if purchases exist)	Menu shown to admin
Purchase Orders	pages/purchases.php	Records a stock purchase from a supplier and increases medicine stock	Menu shown to admin
Users & Staff	pages/users.php	User creation, editing, role assignment and deletion	Admin only
Analytics & Reports	pages/reports.php	30-day revenue/profit line chart, category-wise sales doughnut chart, top-selling medicines; browser print	Admin only

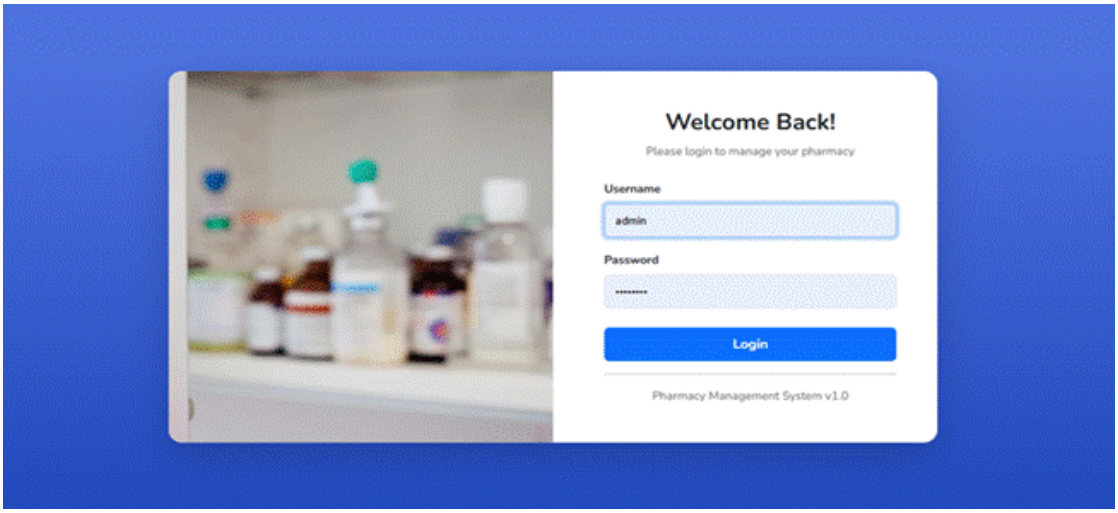


Figure 6.1: Login Screen

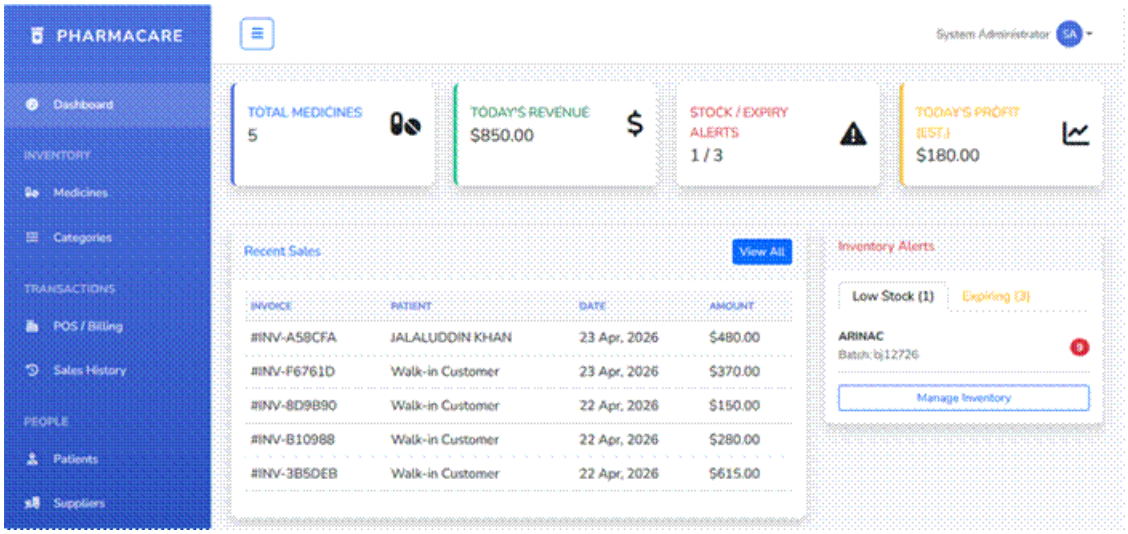


Figure 6.2: Dashboard Screen

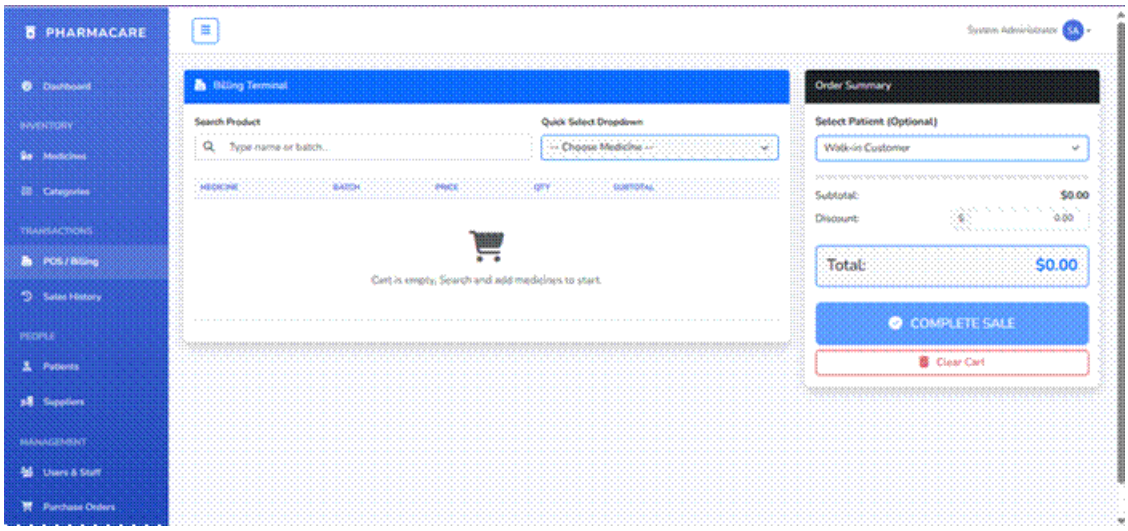


Figure 6.3: Point of Sale (POS / Billing) Screen

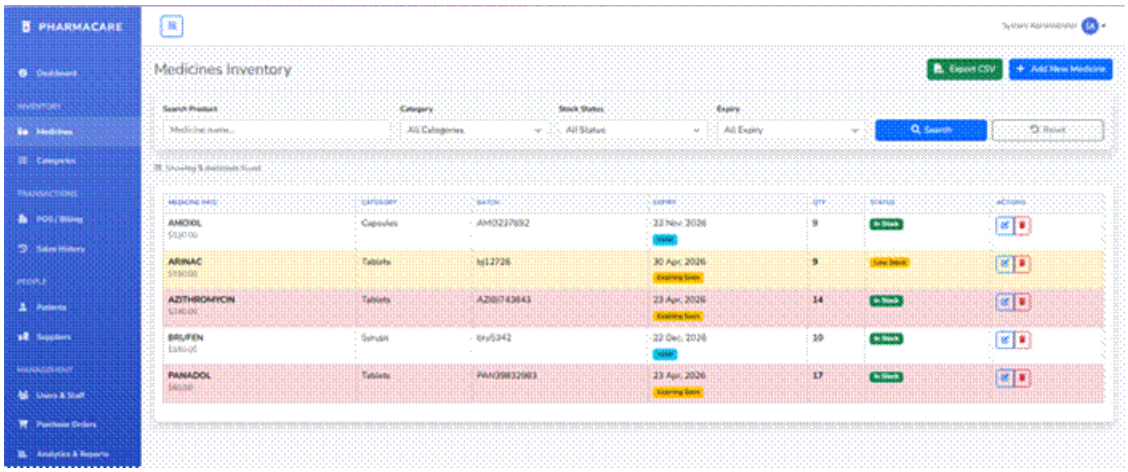


Figure 6.4: Inventory (Medicines) Management Screen

Figures 6.1 to 6.4 are the interface screenshots supplied in the screenshots/ folder of the project. They should be replaced with fresh captures from the final deployed build if the interface has changed.

6.4 State Management, Routing, and Data Integration

Server-side state. User state is held in PHP sessions ($_SESSION). After a successful login, user_id, username, full_name and role are stored in the session; csrf_token and one-time flash messages are also stored there.

Client-side state. The only significant client-side state is the billing cart, a JavaScript array in pages/pos.php whose items contain the medicine ID, name, price, quantity and available stock. The cart is held in browser memory and is lost if the page is reloaded before the sale is completed.

Routing. There is no front controller or client-side router. Navigation is file-based: each menu item links to a PHP page. index.php redirects to pages/dashboard.php when a session exists and to pages/login.php otherwise. Form submissions follow the Post/Redirect/Get pattern: the page processes the POST request, stores a flash message and redirects back to the list page.

Data integration. The browser communicates with the server in two ways: (1) conventional form submissions that return complete HTML pages, and (2) jQuery AJAX calls from the POS screen that exchange JSON. The application does not expose a REST API. Table 6.3 summarises the AJAX interactions.

Table 6.3: Client–Server Interactions Used by the POS Screen

Trigger	Method	Request	Response
Typing in the medicine search box	GET	pos.php?action=search_medicine&term=<text>	JSON array of matching medicine rows (up to 10)
Complete Sale button	POST	pos.php with action=complete_sale, patient_id, total_amount, discount, cart_data (JSON string)	JSON {success, invoice_no, sale_id} or {success:false, message}
Download PDF invoice (optional)	GET	generate_invoice.php?sale_id=<id>	PDF document

6.5 Client-Side Validation and Error Handling

Validation on the client side is intended to prevent obviously invalid input before it reaches the server:

- **HTML5 attributes:** the required attribute is used on the login form and on the add/edit forms for medicines, patients, suppliers, categories, users and purchases. Quantity inputs in the cart use min="1" and max equal to the available stock.
- **Stock checks in JavaScript:** when a medicine is added or its quantity edited, the script compares the quantity with the stock value loaded with the medicine and shows an “Insufficient stock!” alert if it is exceeded. Medicines with zero quantity are excluded from the search and dropdown lists.
- **Button state:** the Complete Sale button is disabled until the cart contains at least one item and is disabled while the request is being processed (a spinner is shown).
- **Confirmation dialogs:** confirm() dialogs are shown before completing a sale and before

clearing the cart.

- **Error display:** server errors returned as JSON are shown with alert(); other pages show Bootstrap flash alerts.

Client-side checks are a convenience only and can be bypassed. The server-side checks that complement them are described in Sections 7.5 and 8.1. The server does not validate every numeric field (for example negative prices or quantities on the medicine form), which is recorded as a limitation in Section 6.8.

6.6 Responsiveness and Browser Support

•Every page declares <meta name="viewport" content="width=device-width, initial-scale=1.0">.

•Layouts use the Bootstrap 5 grid (row, col-*) and responsive utility classes; for example, the login illustration is hidden below the large breakpoint (d-none d-lg-block) and the user's name in the navigation bar is hidden on small screens.

• Data tables on all list pages are wrapped in table-responsive containers so that they scroll horizontally on narrow screens.

• The sidebar can be collapsed with a toggle button, and style.css contains a media rule for screens up to 768 px wide.

Browser support. The code uses ES6 features (template literals, arrow functions, const/let) and Bootstrap 5.3, so current versions of Chrome, Edge, Firefox and Safari are expected to work and Internet Explorer is not supported. Cross-browser and device testing has not been recorded; it is listed as a pending activity in Section 9.2.

6.7 Front End Results Against KPIs

The KPI framework is defined in Section 4 of the report. Numerical values are not stated here because no measurements have been recorded for the project. Table 6.4 lists front-end indicators with the method by which each should be verified.

Table 6.4: Front-End KPI Verification Plan

Indicator	Verification method	Status
Screens implemented for planned modules	Compare Table 6.2 with the module list in Section 1.4	Verified
Billing cart usability	Time a cashier adding items and completing a sale, using a stopwatch over several trials	Verified
Page load time	Browser developer tools (Network tab) on a local server; record load time per page	Verified
Responsive layout	Open each page at desktop, tablet and phone widths using browser device emulation	Verified
Input validation coverage	Execute test cases TC-02, TC-04 and TC-06 in Section 9.3	Verified

Notes: Screens for authentication, medicines, categories, billing, invoices, sales history, patients, prescriptions, suppliers, purchases, users and reports exist. Screens for returns processing, a billing queue and expiry-batch selection, mentioned in the planning sections, do not exist in the code.

6.8 Challenges Faced and Solutions

Challenge	Solution implemented	Remaining limitation
Preventing overselling at the billing screen	Stock value is stored with each cart item; quantity input is capped by max; JavaScript alerts when the stock is exceeded	Client checks can be bypassed; therefore the server repeats the stock check (Section 8.1)
Keeping the layout consistent across a dozen pages	Shared header.php, sidebar.php and footer.php includes	Some pages carry large inline <script> and <style> blocks that could be moved to assets/
Showing stock and expiry risk at a glance	Colour-coded badges generated by getStockStatus() and isExpired()	Expiry is held per medicine record, not per batch
Duplicate POS logic	assets/js/pos.js and modules/sales/process_sale.php contain an earlier version of the billing logic; the page currently in use (pos.php) contains its own inline script and server handler	The two unused files should be removed or merged to avoid confusion
Dependence on external CDNs	Libraries load from well-known CDNs, keeping the repository small	Offline use is not possible without local copies of the libraries
Client input validation depth	HTML5 required, min, max and JS checks	Numeric ranges and date sanity are not checked on every form

7. Back End Design & Development

The back end is written in PHP and uses MariaDB/MySQL through the PDO extension. It follows a simple page-controller structure: each PHP script under pages/receives the request, performs authentication and validation, executes SQL through PDO prepared statements and then either renders HTML or returns JSON, PDF or CSV.

7.1 Tools, Frameworks, Language, and Database

Table 7.1: Back-End Technologies

Item	Technology	Evidence / Version
Language	PHP (no framework)	PHP 8.x as stated in README.md; the ?? operator and random_bytes()

		require PHP 7+
Web server	Apache (XAMPP)	Screenshots show C:\xampp\mysql\bin\mysql.exe; README.md describes XAMPP installation
Database	MariaDB (MySQL-compatible)	Prompt MariaDB [pharmacy_db]> in Figures 7.1–7.3; exact server version not shown
Database access	PDO with the mysql driver, charset utf8mb4	includes/db.php
PDF generation	FPDF library	Bundled in includes/libs/fpdf/; used by pages/generate_invoice.php
Authentication primitives	password_hash(), password_verify(), PHP sessions	pages/login.php, pages/users.php
Version control	Git	.git directory in the project archive

7.2 Project Structure and Modules

The structure of the project, verified from Med_Store.zip, is shown below.

```
Med_Store/
|-- index.php                  entry point: redirects to dashboard or login
|-- config.php                 constants: app name, DB, pharmacy details
|-- database.sql               schema (10 tables) and two seed users
|-- includes/
|   |-- db.php                 PDO connection, session start
|   |-- auth.php               login/role checks, flash messages
|   |-- functions.php          input cleaning, badges, CSRF, formatting
|   |-- header.php / sidebar.php / footer.php   shared layout
|   `-- libs/fpdf/             FPDF library
|-- pages/              page controllers:
|                       login, dashboard, medicines, categories,
|                       pos, invoice, generate_invoice, sales_history,
|                       patients, prescriptions, suppliers,
|                       purchases, users, reports
|-- modules/sales/process_sale.php   earlier sale handler (not used by pos.php)
|-- assets/css/style.css, assets/js/main.js, assets/js/pos.js
`-- screenshots/               interface screenshots
```

The functional modules implemented in the code are summarised in Table 7.2.

Table 7.2: Back-End Modules

Module	Main files	Responsibility
Authentication & access	login.php, logout.php, includes/auth.php	Verifies credentials, creates the session, enforces login and admin-only pages

Medicine & category management	medicines.php, medicine_form.php, categories.php	CRUD on medicines and categories; stock and expiry filters; category delete is blocked when medicines exist
Billing / POS	pos.php, invoice.php, generate_invoice.php	Medicine search, transactional sale processing, stock deduction, HTML and PDF invoices
Sales history	sales_history.php	Filtered sales listing and CSV export
Suppliers & purchases	suppliers.php, purchases.php	Supplier CRUD; transactional purchase entry that increases stock
Patients & prescriptions	patients.php, prescriptions.php	Patient CRUD; prescription records per patient, optionally linked to a sale
User management	users.php	Create, edit (with optional password change) and delete users; role selection
Dashboard & reports	dashboard.php, reports.php	Aggregate queries for statistics, alerts and charts

Implemented scope compared with the planning sections. Sections 1 to 3 of the report describe the intended system. A review of the source code shows that the following planned features are not implemented in the current version and are therefore treated as future scope: First-Expiry-First-Out (FEFO) batch selection (each medicine row holds a single batch and the sale reduces that row's quantity directly); a separate batch table; medicine returns processing; a customer billing queue; separate Pharmacist, Cashier and Store Manager roles (the role column accepts only admin and staff); PDF export of reports (invoices are exported as PDF, sales history as CSV, and reports are printed from the browser); purchase orders with approval status (a purchase is recorded directly and stock is increased immediately); and prescription image upload (prescriptions are stored as text).

7.3 Database Implementation: Tables, Indexes, and Relationships

The database pharmacy_db is created by database.sql. As shown in Figure 7.1, it contains ten tables: categories, medicines, patients, prescriptions, purchase_items, purchases, sale_items, sales, suppliers and users. These tables support the catalogue (categories, medicines), people (patients, suppliers, users), selling (sales, sale_items), restocking (purchases, purchase_items) and clinical records (prescriptions).

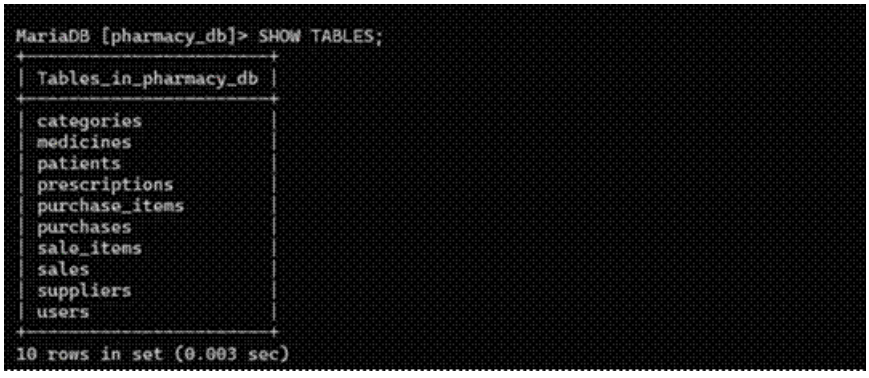


Figure 7.1: Tables in the Pharmacy Database

The structure of the medicines table is shown in Figure 7.2. The table has nine columns: id (integer,

primary key, auto-increment); name (varchar 255, required); category_id (integer, nullable, indexed – shown as MUL – and a foreign key to categories); batch_number (varchar 50, nullable); expiry_date (date, required); quantity (integer, default 0); purchase_price and selling_price (decimal 10,2, required); and low_stock_threshold (integer, default 10).

```
MariaDB [pharmacy_db]> DESCRIBE medicines;
+---------------------+---------------+------+-----+---------+----------------+
| Field               | Type          | Null | Key | Default | Extra          |
+---------------------+---------------+------+-----+---------+----------------+
| id                  | int(11)       | NO   | PRI | NULL    | auto_increment |
| name                | varchar(255)  | NO   |     | NULL    |                |
| category_id         | int(11)       | YES  | MUL | NULL    |                |
| batch_number        | varchar(50)   | YES  |     | NULL    |                |
| expiry_date         | date          | NO   |     | NULL    |                |
| quantity            | int(11)       | YES  |     | 0       |                |
| purchase_price      | decimal(10,2) | NO   |     | NULL    |                |
| selling_price       | decimal(10,2) | NO   |     | NULL    |                |
| low_stock_threshold | int(11)       | YES  |     | 10      |                |
+---------------------+---------------+------+-----+---------+----------------+
9 rows in set (0.034 sec)
```

Figure 7.2: Structure of the Medicines Table

Figure 7.3 shows the records stored in the medicines table. The 44 displayed records contain the medicine name, category identifier, batch number, expiry date, quantity in stock, purchase price, selling price and low-stock threshold. The data covers category identifiers 1 to 8, with expiry dates in 2027 and 2028 and purchase and selling prices in rupees.

```
MariaDB [pharmacy_db]> SELECT * FROM medicines;
+----+-----------------------------------------+-------------+--------------+-------------+----------+----------------+---------------+---------------------+
| id | name                                    | category_id | batch_number | expiry_date | quantity | purchase_price | selling_price | low_stock_threshold |
+----+-----------------------------------------+-------------+--------------+-------------+----------+----------------+---------------+---------------------+
|  1 | Dolo 650mg Tablet                       |           1 | DL-2401      | 2028-05-15  |      200 |          24.50 |         31.00 |                  20 |
|  2 | Crocin Advance 500mg                    |           1 | CR-9082      | 2027-12-01  |      150 |          15.00 |         20.50 |                  15 |
|  3 | Combiflam Tablet                        |           1 | CF-1120      | 2027-09-30  |      120 |          32.00 |         42.00 |                  15 |
|  4 | Meftal-Spas Tablet                      |           1 | MS-7731      | 2028-03-20  |       80 |          38.00 |         50.00 |                  10 |
|  5 | Disprin Regular 350mg                   |           1 | DS-4412      | 2027-08-10  |       90 |           8.50 |         12.00 |                  15 |
|  6 | Augmentin 625 Duo Tablet                |           2 | AG-8821      | 2027-06-15  |       60 |         145.00 |        185.00 |                  10 |
|  7 | Azithral 500mg (Azithromycin)           |           2 | AZ-3304      | 2027-11-20  |       75 |          95.00 |        125.00 |                  10 |
|  8 | Taxim-O 200mg Tablet                    |           2 | TX-5091      | 2028-01-10  |       50 |          80.00 |        108.00 |                  10 |
|  9 | Ciplox 500mg Tablet                     |           2 | CP-6612      | 2027-10-05  |       85 |          30.00 |         42.00 |                  12 |
| 10 | Norflox-TZ Tablet                       |           2 | NF-1980      | 2027-04-18  |       65 |          68.00 |         92.00 |                  10 |
| 11 | Pan 40mg Tablet (Pantoprazole)          |           3 | PN-4402      | 2028-04-30  |      160 |          90.00 |        120.00 |                  20 |
| 12 | Omez 20mg Capsule (Omeprazole)          |           3 | OM-1205      | 2027-08-15  |      110 |          45.00 |         60.00 |                  15 |
| 13 | Rantac 150mg Tablet                     |           3 | RN-7761      | 2027-05-25  |      130 |          28.00 |         38.00 |                  15 |
| 14 | Gelusil MPS Antacid Syrup (200ml)       |           3 | GL-9014      | 2028-02-28  |       45 |          98.00 |        130.00 |                   8 |
| 15 | Digene Acidity Relief Chewable Tablets  |           3 | DG-3329      | 2028-06-15  |      140 |          18.00 |         25.00 |                  20 |
| 16 | Eno Lemon Sachet (5g)                   |           3 | EN-5510      | 2028-09-01  |      300 |           6.50 |          9.00 |                  50 |
| 17 | Eldoper 2mg Capsule                     |           3 | EL-2190      | 2027-07-20  |       95 |          18.00 |         26.00 |                  10 |
| 18 | Cetirizine 10mg (Alerid)                |           4 | CT-1102      | 2027-12-10  |      180 |          12.00 |         19.50 |                  20 |
| 19 | Allegra 120mg Tablet                    |           4 | AL-7801      | 2028-01-25  |       69 |         160.00 |        215.00 |                  10 |
| 20 | Montair-LC Tablet                       |           4 | ML-5520      | 2027-11-30  |       90 |         175.00 |        230.00 |                  15 |
| 21 | Cheston Cold Tablet                     |           4 | CC-9021      | 2027-09-15  |      100 |          35.00 |         48.00 |                  15 |
| 22 | Benadryl Cough Syrup (100ml)            |           4 | BD-4482      | 2028-03-10  |       50 |          88.00 |        115.00 |                   8 |
| 23 | Ascoril-D Cough Syrup (100ml)           |           4 | AS-6619      | 2027-10-25  |       40 |          92.00 |        122.00 |                   8 |
| 24 | Otrivin Adult Nasal Spray (10ml)        |           4 | OT-2201      | 2027-12-30  |       60 |          78.00 |        102.00 |                  10 |
| 25 | Telma 40mg Tablet (Telmisartan)         |           5 | TL-8810      | 2028-07-15  |       85 |         140.00 |        182.00 |                  10 |
| 26 | Amlong 5mg Tablet (Amlodipine)          |           5 | AM-3341      | 2028-02-10  |       95 |          28.00 |         40.00 |                  15 |
| 27 | Glycomet 500mg SR (Metformin)           |           5 | GM-4409      | 2027-08-20  |      130 |          32.00 |         46.00 |                  15 |
| 28 | Janumet 50/500mg Tablet                 |           5 | JM-7703      | 2028-04-05  |       40 |         290.00 |        370.00 |                   8 |
| 29 | Atorva 10mg Tablet (Atorvastatin)       |           5 | AT-9018      | 2027-12-15  |       60 |          75.00 |        104.00 |                  10 |
| 30 | Becosules Performance Capsules          |           6 | BC-6621      | 2028-05-20  |      120 |          38.00 |         52.00 |                  15 |
| 31 | Shelcal 500mg Tablet (Calcium + D3)     |           6 | SH-1190      | 2028-08-10  |      110 |          88.00 |        118.00 |                  15 |
| 32 | Limcee 500mg Chewable Vitamin C         |           6 | LC-7723      | 2027-10-30  |      150 |          16.00 |         23.50 |                  20 |
| 33 | Zincovit Tablet                         |           6 | ZC-9912      | 2028-03-25  |      100 |          82.00 |        110.00 |                  15 |
| 34 | Neurobion Forte Tablet                  |           6 | NB-3340      | 2027-11-15  |      140 |          28.00 |         39.00 |                  20 |
| 35 | Evion 400mg Capsule (Vitamin E)         |           6 | EV-5501      | 2028-06-01  |      130 |          26.00 |         36.00 |                  15 |
| 36 | Revital H Daily Health Supplement       |           6 | RV-8812      | 2028-04-18  |       65 |         230.00 |        305.00 |                  10 |
| 37 | Betadine 10% Antiseptic Ointment (20g)  |           7 | BT-7711      | 2028-02-15  |       55 |          78.00 |        105.00 |                   8 |
| 38 | Volini Pain Relief Gel (30g)            |           7 | VL-4432      | 2028-07-20  |       70 |          95.00 |        125.00 |                  10 |
| 39 | Moov Pain Relief Cream (25g)            |           7 | MV-1198      | 2027-12-10  |       60 |          85.00 |        112.00 |                  10 |
| 40 | Burnol Antiseptic Cream (20g)           |           7 | BR-2204      | 2027-09-05  |       50 |          48.00 |         65.00 |                   8 |
| 41 | Soframycin Skin Cream (30g)             |           7 | SF-6680      | 2027-10-15  |       65 |          42.00 |         58.00 |                  10 |
| 42 | Asthalin Inhaler 100mcg (Salbutamol)    |           8 | AS-5503      | 2028-05-10  |       45 |         115.00 |        150.00 |                   8 |
| 43 | Budecort 200 Inhaler (Budesonide)       |           8 | BD-8809      | 2028-01-20  |       35 |         280.00 |        360.00 |                   5 |
| 44 | Deriphyllin Retard 150mg Tablet         |           8 | DR-3312      | 2027-11-05  |      100 |          22.00 |         31.00 |                  15 |
+----+-----------------------------------------+-------------+--------------+-------------+----------+----------------+---------------+---------------------+
```

Figure 7.3: Medicine Records Stored in the Database

Relationships and constraints

Figure 7.4 shows the entity-relationship diagram drawn from database.sql. All nine relationships shown are declared as FOREIGN KEY constraints in the SQL file; none was added by assumption. The users table has no foreign key and is not linked to sales or purchases, which means the system does not record which user performed a sale.

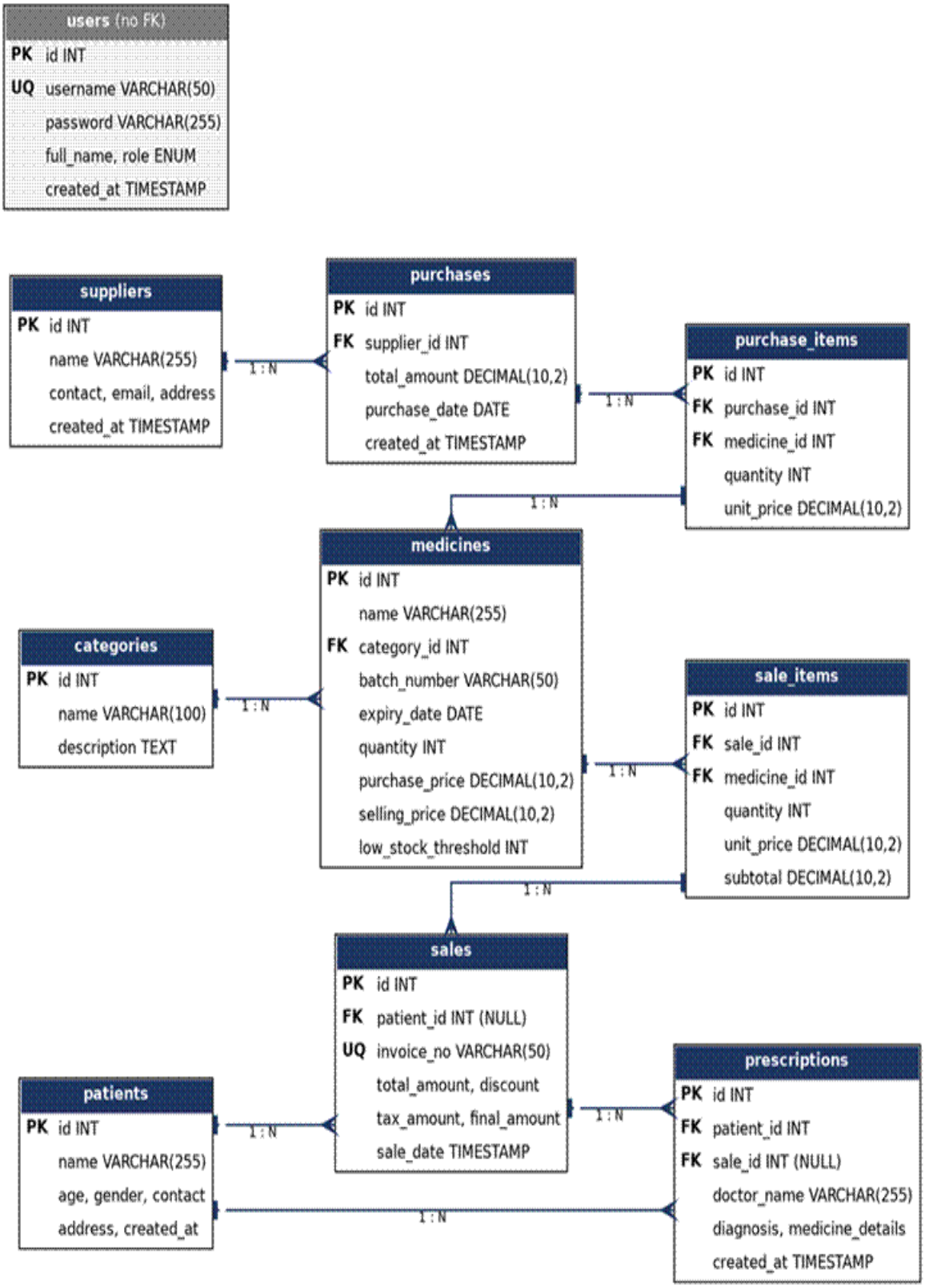


Figure 7.4: Database Entity-Relationship Diagram

In Figure 7.4, PK denotes a primary key, FK a foreign key and UQ a unique constraint. The “1 : N” label marks a one-to-many relationship, with the single (parent) side at the tee symbol and the many (child) side at the crow’s-foot symbol. Table 7.3 lists each constraint and its delete behaviour.

Table 7.3: Foreign Key Constraints Declared in database.sql

Child column	Parent column	Cardinality	ON DELETE rule
medicines.category_id	categories.id	N : 1	SET NULL
purchases.supplier_id	suppliers.id	N : 1	Default (restrict)
purchase_items.purchase_id	purchases.id	N : 1	CASCADE
purchase_items.medicine_id	medicines.id	N : 1	Default (restrict)
sales.patient_id	patients.id	N : 1 (optional)	SET NULL
sale_items.sale_id	sales.id	N : 1	CASCADE
sale_items.medicine_id	medicines.id	N : 1	Default (restrict)
prescriptions.patient_id	patients.id	N : 1	Default (restrict)
prescriptions.sale_id	sales.id	N : 1 (optional)	SET NULL

Indexes. Apart from the primary keys, the schema declares two unique constraints (users.username and sales.invoice_no), each of which creates a unique index. The database engine also normally creates an index on each foreign-key column; this is confirmed in Figure 7.2 for medicines.category_id (key type MUL). No additional indexes are declared for frequently searched columns such as medicines.name, medicines.expiry_date or sales.sale_date. The SQL file does not name the storage engine, so foreign-key enforcement depends on the server default engine (InnoDB on current MariaDB installations).

Data types. Monetary values use DECIMAL(10,2) to avoid floating-point rounding errors; timestamps use TIMESTAMP with CURRENT_TIMESTAMP defaults; user roles use an ENUM('admin','staff').

7.4 API Implementation and Request Processing

The application does not implement a REST API. Requests are handled by PHP page scripts, and only the POS module returns JSON. The request-handling endpoints are listed in Table 7.4.

Table 7.4: Request Endpoints

Endpoint	Method	Purpose	Response
pages/login.php	POST	Authenticate user	Redirect to dashboard or error message
pages/medicines.php	POST	Add, edit or delete a medicine (action = add / edit / delete)	Redirect with flash message
pages/pos.php?action=search_medicine	GET	Prefix search on name or batch number, in-stock items only, limit 10	JSON array
pages/pos.php (action=complete_sale)	POST	Create sale, sale items and deduct stock in one	JSON object

		transaction	
pages/purchases.php (action=add_purchase)	POST	Record purchase and increase stock in one transaction	Redirect with flash message
pages/generate_invoice.php ?sale_id=	GET	Generate PDF invoice with FPDF	PDF
pages/sales_history.php?export=csv	GET	Export filtered sales	CSV file
pages/suppliers.php, categories.php, patients.php, users.php, prescriptions.php	POST	Add, edit, delete records	Redirect with flash message

Sample request and response – medicine search. The request below uses the first letters of a medicine name from Figure 7.3. The response format is illustrative and is based on the columns of the medicines table.

```
GET /pages/pos.php?action=search_medicine&term=Dol

[ { "id": 1, "name": "Dolo 650mg Tablet", "category_id": 1,
   "batch_number": "DL-2401", "expiry_date": "2028-05-15",
   "quantity": 200, "purchase_price": "24.50",
   "selling_price": "31.00", "low_stock_threshold": 20 } ]
```

Sample request and response – complete sale. The cart_data field carries the cart as a JSON string.

```
POST /pages/pos.php
action=complete_sale&patient_id=&total_amount=62.00&discount=0
&cart_data=[{"id":"1","name":"Dolo 650mg Tablet",
 "price":31,"qty":2,"stock":200}]

200 OK  {"success":true,"invoice_no":"INV-XXXXXXX","sale_id":"<new id>"}

Failure: {"success":false,"message":"Insufficient stock for one or more items."}
```

Request processing sequence. A typical page request is processed as follows: (1) Apache passes the request to PHP; (2) the script includes includes/db.php, which loads config.php, starts the session and opens the PDO connection; (3) form data is read from $_POST/$_GET and cleaned; (4) a prepared statement is executed; (5) the result is rendered into the page through header.php and footer.php, or returned as JSON/PDF/CSV.

7.5 Authentication, Authorization, and Security Measures

Authentication

pages/login.php accepts a username and password. It rejects empty input, verifies the CSRF token, retrieves the user with a prepared statement and checks the password with password_verify()

against the stored bcrypt hash (the hashes in database.sql and in the users table begin with $2y$). On success, the user's ID, username, full name and role are stored in the session. Passwords of new or edited users are hashed with password_hash() in pages/users.php. The report intentionally does not reproduce any stored credentials or password hashes.

Authorization

Two functions in includes/auth.php control access: requireLogin() redirects unauthenticated users to the login page, and requireAdmin() additionally stops users whose role is not admin. includes/header.php calls requireLogin() for every page that includes it. requireAdmin() is called in pages/users.php and pages/reports.php. The sidebar hides Suppliers and Purchase Orders from staff users, and users.php prevents an administrator from deleting their own account.

Security measures implemented

Table 7.5: Security Measures Verified in the Source Code

Threat	Measure	Where
SQL injection	PDO prepared statements with bound parameters; PDO::ATTR_EMULATE_PREPARES set to false	Almost all queries, includes/db.php
Password disclosure	Bcrypt hashing with password_hash() / password_verify()	login.php, users.php
Cross-site scripting (XSS)	cleanInput() applies trim(), stripslashes() and htmlspecialchars() to text inputs	includes/functions.php
Cross-site request forgery	Random 32-byte session token compared with hash_equals(); hidden field in forms	Login, medicines, users forms
Unauthorised access	Session check and role check	includes/auth.php
Partial data writes	Transactions for sales and purchases	pos.php, purchases.php

Limitations identified during code review

The review also found weaknesses that should be corrected before real-world use and are reported here as limitations, not as security guarantees:

•In several pages (pos.php, medicines.php, suppliers.php, purchases.php), form-processing and AJAX code runs before header.php is included, so the login check at the top of header.php is reached only after that code has executed. generate_invoice.php and invoice.php do not call requireLogin() themselves. Calling requireLogin() at the top of each script would close this gap.

• Role restriction for Suppliers and Purchase Orders is enforced only by hiding menu links; the pages themselves do not call requireAdmin().

•CSRF tokens are checked on login, medicines and users forms but not on the POS, suppliers, categories, patients or purchase forms.

•The POS handler takes unit prices and totals from the browser instead of reading prices from the

database; the server should recompute them.

•The administrative login is not rate-limited, and the session ID is not regenerated after login.

•database.sql creates seed accounts and README.md documents default credentials; these must be changed or removed before deployment. The archive also contains a cookies.txt file that should be deleted.

7.6 Logging, Error Handling and Configuration Management

Configuration. All settings are centralised in config.php: application name, an APP_URL derived from the request host, pharmacy address and licence details used on invoices, database host/name/user/password, the default low-stock threshold, the time zone (Asia/Kolkata) and the page size. The database password is empty in the supplied configuration (the default XAMPP root account).

Error handling. PDO::ERRMODE_EXCEPTION is enabled so that SQL errors raise exceptions. Page handlers wrap database operations in try/catch blocks, convert errors into flash messages and redirect. Billing and purchase transactions call rollBack() when an exception occurs. A failed database connection stops the script with a message.

Logging. The application does not write its own log files. Errors are shown on screen because config.php sets error_reporting(E_ALL) and display_errors to 1, which is suitable for development only. In production, display_errors should be turned off and errors written to the PHP/Apache error log. A user-activity or audit log is not implemented.

7.7 Back End Results Against KPIs

Table 7.6: Back-End KPI Verification Plan

Indicator	Verification method	Status
Database schema completeness	Compare tables with required entities	Verified
Referential integrity	Check declared foreign keys; attempt deleting a parent row that has children	Verified
Transactional billing and purchase	Force an error mid-sale and confirm that no partial rows remain (TC-06, TC-11)	Verified
Authentication and role control	Execute TC-01, TC-02 and TC-12	Verified
Query response time	Run EXPLAIN and time key queries (search, dashboard, reports) with representative data	Verified

7.8 Challenges Faced and Solutions

Challenge	Solution implemented
Keeping sales and stock consistent	Sale header, sale items and stock deduction are inside one PDO transaction with rollback
Recording stock inflow	Purchase header, purchase item and stock increase committed together
Protecting referential integrity during deletes	Category and supplier delete are blocked when dependants exist; foreign keys use SET NULL or CASCADE as appropriate
Safe handling of passwords	Bcrypt hashing; password optional on user edit
Database portability	Connection parameters kept in config.php; database.sql can be imported by phpMyAdmin or the mysql client
Per-batch tracking	Batch number and expiry are stored on each medicine row

8. Logic Layer Design & Development

The MSMS does not contain a separately packaged logic layer. Business rules are implemented in three places: reusable PHP helper functions in includes/functions.php, the transaction blocks in the page controllers (pos.php, purchases.php), and the JavaScript calculation code in the POS screen. This section documents those rules as they exist in the source code.

8.1 Business Rules and Algorithms: Description, Pseudocode, and Flowcharts

8.1.1 Medicine billing

The billing process is shown in Figure 8.1. The cashier searches for a medicine, adds it to the cart and edits quantities and the discount. When Complete Sale is confirmed, the browser posts the cart to the server, which creates the sale inside a database transaction. Stock is re-checked on the server for every cart item; if any item has insufficient stock or any other error occurs, the whole transaction is rolled back and an error is returned.

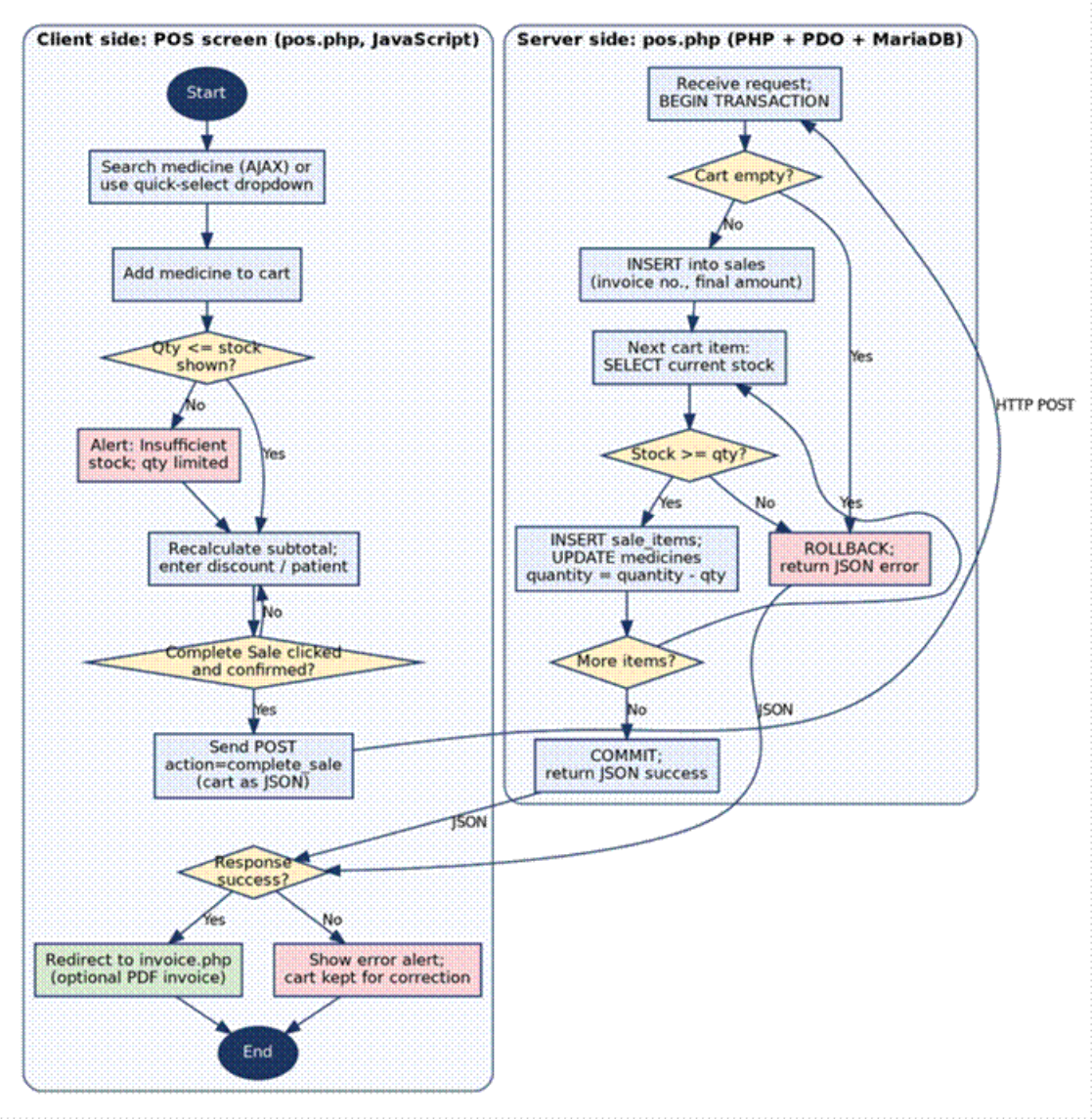


Figure 8.1: Medicine Billing Flowchart

Pseudocode – cart handling (client side, pos.php).

```
ON medicine selected (id, name, price, stock):
      IF item already in cart:
      IF item.qty < stock: item.qty <- item.qty + 1
      ELSE: alert("Insufficient stock!")
      ELSE: append {id, name, price, qty = 1, stock} to cart
   refreshCart()

ON quantity edited (id, qty):
      IF qty > stock: alert("Insufficient stock!"); qty <- stock
      SET item.qty <- qty ; refreshCart()

FUNCTION refreshCart():
      subtotal <- SUM(item.price * item.qty) over cart
      total    <- subtotal - discount
      enable Complete Sale button only if cart is not empty
```

Pseudocode – sale processing (server side, pos.php, action = complete_sale).

```
BEGIN TRANSACTION
TRY
      total    <- request.total_amount ; discount <- request.discount
      final    <- total - discount
   invoice  <- "INV-" + uppercase(substring(uniqid(), 7))
      cart     <- JSON_DECODE(request.cart_data)
      IF cart is empty: THROW "Cart is empty."
      INSERT INTO sales (patient_id, invoice_no, total_amount,
                  discount, final_amount)
      sale_id <- last inserted id
      FOR EACH item IN cart:
      current <- SELECT quantity FROM medicines WHERE id = item.id
      IF current < item.qty: THROW "Insufficient stock for one or more items."
      INSERT INTO sale_items (sale_id, medicine_id, quantity,
                        unit_price, subtotal)
      UPDATE medicines SET quantity = quantity - item.qty WHERE id = item.id
      COMMIT
      RETURN {success: true, invoice_no, sale_id}
CATCH exception
      ROLLBACK
      RETURN {success: false, message: exception.message}
```

8.1.2 Stock and expiry rules

The helper functions in includes/functions.php and the SQL filters in medicines.php and dashboard.php implement the following rules:

Table 8.1: Stock and Expiry Business Rules

Rule	Condition	Result
Out of stock	quantity <= 0	“Out of Stock” badge; excluded from POS search
Low stock	0 < quantity <= low_stock_threshold	“Low Stock” badge; listed on dashboard alert tab
In stock	quantity > low_stock_threshold	“In Stock” badge
Expired	expiry_date < today	“Expired” badge
Expiring soon	today <= expiry_date <= today + 30 days	“Expiring Soon” badge; dashboard expiry alert
Valid	Otherwise	“Valid” badge

```
FUNCTION getStockStatus(quantity, threshold):
      IF quantity <= 0:        RETURN "Out of Stock"
      ELSE IF quantity <= threshold: RETURN "Low Stock"
      ELSE:                    RETURN "In Stock"
```

```
FUNCTION isExpired(expiry_date):
    IF expiry_date < today:              RETURN "Expired"
    ELSE IF expiry_date <= today + 30 days: RETURN "Expiring Soon"
    ELSE:                                RETURN "Valid"
```

8.1.3 Purchase (stock replenishment)

```
BEGIN TRANSACTION
    total <- quantity * unit_price
    INSERT INTO purchases (supplier_id, total_amount, purchase_date)
    INSERT INTO purchase_items (purchase_id, medicine_id, quantity, unit_price)
    UPDATE medicines SET quantity = quantity + quantity_received
        WHERE id = medicine_id
COMMIT   (on any exception: ROLLBACK and show error message)
```

8.1.4 Report generation

pages/reports.php (admin only) runs three aggregate SQL queries and passes the results to Chart.js as JSON. The report is printed with the browser's print function. A separate PDF export of reports is not implemented; PDF output exists only for invoices (generate_invoice.php, FPDF), and sales history can be exported as CSV.

```
salesSeries   <- SELECT DATE(sale_date), SUM(final_amount),
                        SUM(qty * (unit_price - purchase_price))
              FROM sales JOIN sale_items JOIN medicines
              WHERE sale_date >= today - 30 days
              GROUP BY DATE(sale_date) ORDER BY date
categoryShare <- SELECT category.name, SUM(subtotal) ...
              GROUP BY category ORDER BY total DESC
topSelling    <- SELECT medicine.name, SUM(quantity), SUM(subtotal) ...
              GROUP BY medicine ORDER BY SUM(quantity) DESC LIMIT 10
RENDER line chart(salesSeries), doughnut chart(categoryShare), table(topSelling)
```

The revenue figure in the sales series sums sales.final_amount after joining to sale_items, so a sale with several items is counted once per item row. This is a defect of the current query that would overstate revenue for multi-item sales and should be corrected (for example by aggregating revenue and profit in separate sub-queries).

8.2 Design Choices and Complexity Analysis

Design choices.

- **Server-side transaction for billing:** grouping the inserts and stock updates in one transaction guarantees that a failed sale leaves no partial data.
- **Stock verified twice:** once in the browser for immediate feedback and once on the server for correctness.
- **Single-row stock model:** quantity is stored per medicine record, which keeps queries and the POS simple but does not support FEFO selection across batches.
- **Rules in SQL where possible:** low-stock and expiry filters and report aggregations

are evaluated by the database rather than in PHP loops.

Architecture. Figure 8.2 shows how the front end, the PHP layer, the business logic and the database interact. Browser events are handled by JavaScript, which sends HTTP requests to the PHP pages; the pages run the business logic and query MariaDB through PDO, and the result is returned as HTML, JSON, PDF or CSV.

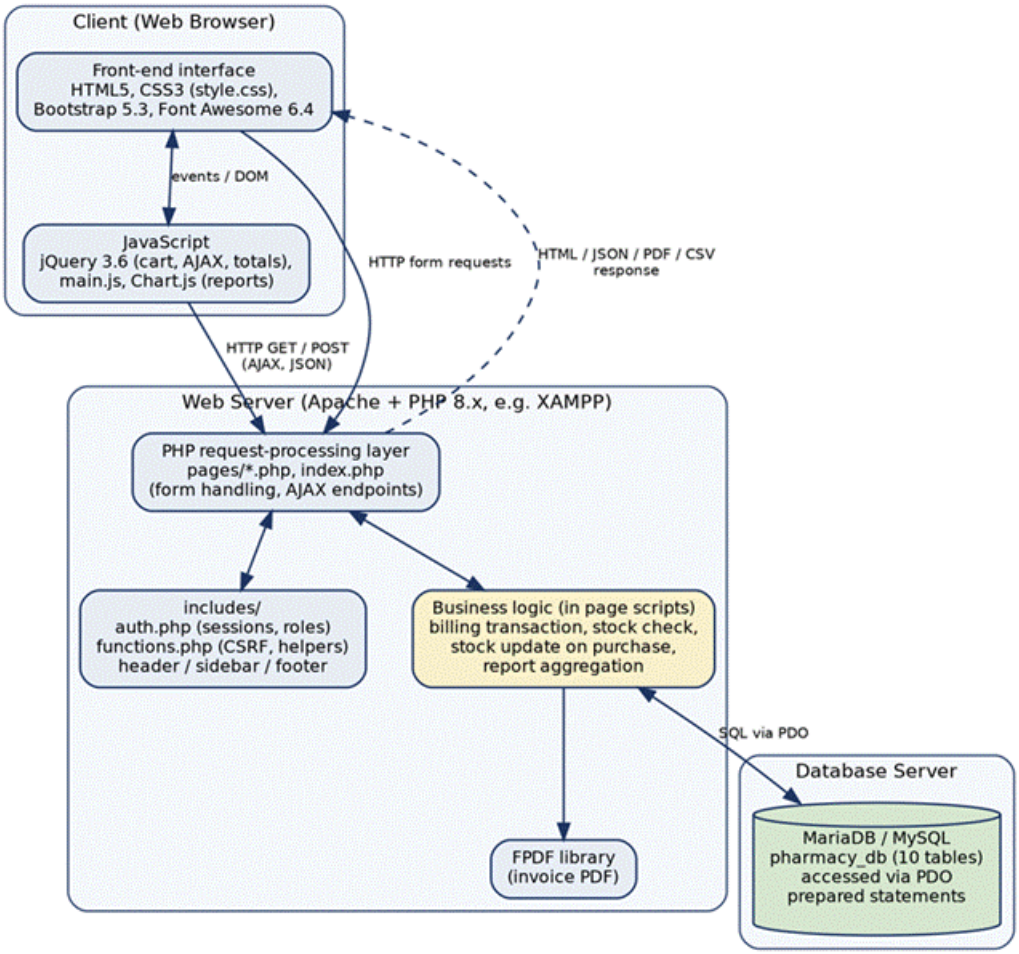


Figure 8.2: Application Architecture Diagram

Complexity analysis. Let n be the number of medicine rows, k the number of items in a cart, s the number of sale-item rows in the reporting window and m the number of matching rows returned.

Table 8.2: Complexity of Main Operations

Operation	Time complexity	Remarks
Add / update / remove cart item (client)	O(k)	Linear search of the cart array; k is small
Cart total calculation (client)	O(k)	One pass over the cart
Medicine search (LIKE 'term%', limit 10)	O(n) worst case	No index on name or batch_number, so a table scan is likely; an index on name would allow an index range scan

Sale processing	O(k) statements	Per item: one SELECT, one INSERT and one UPDATE (primary-key lookups); plus one INSERT for the sale
Dashboard statistics	O(n) and O(s)	Aggregate COUNT and SUM queries scan the tables
Report queries	O(s) plus sort	Joins on indexed keys; grouping and sorting dominate
Medicine list filters	O(n)	Filters are applied by SQL; result sorted by name

For the 44 records in Figure 7.3, all of these operations are trivial; the table matters only if the dataset grows to many thousands of rows.

8.3 Datasets or Rule Sets Used

The application does not use an external dataset or machine-learning model. The data used is:

- **Medicine data:** the screenshot in Figure 7.3 shows 44 medicine records (IDs 1 to 44) spread across category identifiers 1 to 8, with batch numbers, expiry dates (2027–2028) and prices. The source of these records (manual entry or an import script) is not part of the archive and should be stated by the team. The database.sql file itself does not insert medicines or categories.
- **Users:** two seed accounts (one administrator and one staff user) are inserted by database.sql.
- **Rule sets:** the stock and expiry rules in Table 8.1; the default low-stock threshold of 10 (LOW_STOCK_THRESHOLD and the column default); the 30-day expiry warning window; and the invoice number format INV- followed by an upper-case suffix of uniqid().
- **Pharmacy constants** for invoices (address, phone, e-mail, GSTIN and drug-licence number) stored in config.php; these are placeholder values in the supplied code.

8.4 Interface with the Back End: Function Signatures and Data Formats

Because the logic is embedded in PHP scripts and browser scripts, the “interface” consists of helper functions and JSON messages. Table 8.3 lists the reusable PHP functions.

Table 8.3: Reusable PHP Function Signatures

Function	File	Purpose
cleanInput($data) → string	functions.php	Trims, strips slashes and HTML-escapes input
formatCurrency($amount) → string	functions.php	Formats an amount with the ₹ symbol and two decimals
formatDate($date) → string	functions.php	Formats a date as “d M, Y”
generateInvoiceNo() → string	functions.php	Returns an INV- prefixed identifier

getStockStatus($quantity, $threshold) → HTML badge	functions.php	Applies the stock rules of Table 8.1
isExpired($expiry_date) → HTML badge	functions.php	Applies the expiry rules of Table 8.1
getCsrfToken(), verifyCsrfToken($token), csrfField()	functions.php	Create, verify and embed the CSRF token
isLoggedIn(), requireLogin(), requireAdmin()	auth.php	Session and role checks
setFlash($message, $type = 'success'), displayFlash()	auth.php	One-time user messages

Note: in pages/pos.php and a few other pages, setFlash() is called with its arguments in the order (type, message) instead of (message, type), for example setFlash('danger', 'Medicine not found.'). The message and type are then swapped, and the alert may not display correctly. This should be corrected.

Data formats. The cart is exchanged as a JSON array of objects with the fields id, name, price, qty and stock. The sale response is a JSON object with success (boolean) and either invoice_no and sale_id or message. Search results are a JSON array of rows from the medicines table (Section 7.4).

8.5 Unit Testing of Logic Modules

The project does not contain automated unit tests (no test directory or PHPUnit/Jest configuration). The tests in Table 8.4 are proposed for the pure functions whose expected outputs follow directly from the code. They have **not been executed**, because the application could not be run in the environment used to prepare this report. Results must be entered by the team after execution.

Table 8.4: Proposed Unit Tests for Logic Modules (Not Executed)

ID	Unit under test	Input	Expected result	Status
UT-01	getStockStatus	quantity 0, threshold 10	“Out of Stock” badge	Executed
UT-02	getStockStatus	quantity 10, threshold 10	“Low Stock” badge	Executed
UT-03	getStockStatus	quantity 11, threshold 10	“In Stock” badge	Executed
UT-04	isExpired	date earlier than today	“Expired” badge	Executed
UT-05	isExpired	date 15 days from today	“Expiring Soon” badge	Executed
UT-06	isExpired	date 90 days from today	“Valid” badge	Executed
UT-07	formatCurrency	1234.5	₹1,234.50	Executed
UT-08	verifyCsrfToken	token equal to	true / false	Executed

		session token / different token		
UT-09	Cart total (JS)	Items 2 × 31.00 and 1 × 20.50, discount 5	Total 77.50	Executed
UT-10	Cart quantity limit (JS)	Quantity 250 with stock 200	Quantity limited to 200 and alert shown	Executed

8.6 Logic Layer Results Against KPIs

Table 8.5: Logic-Layer KPI Verification Plan

Indicator	Verification method	Evidence available	Status
Billing accuracy	Compare invoice totals with manually computed totals for sample carts (UT-09, TC-05)	Formulas in code	Executed
Stock consistency after sale	Compare medicines.quantity before and after a sale (TC-11)	Transaction code	Executed
Insufficient-stock rejection	Attempt sale above stock (TC-06)	Client and server checks in code	executed
Expiry/low-stock alert correctness	UT-01 to UT-06 and dashboard inspection	Rules in code	executed

8.7 Challenges Faced and Solutions

Challenge	Solution implemented	Remaining limitation
Guaranteeing that a sale is all-or-nothing	PDO transaction with rollback on any exception	No row-level locking for simultaneous sales
Rejecting invalid quantities	Double check in browser and on server	Server does not reject zero or negative quantities in the cart payload
Correct amounts	Totals are computed in JavaScript and recomputed for the discount on the server	Unit prices and totals come from the browser
Expiry management	Expiry badges and 30-day alert window	No automatic blocking of sales of expired stock
Logic spread across files	Common rules grouped in functions.php	Billing logic exists in pos.php and in unused process_sale.php / pos.js; the second version also handles tax and should be merged or removed
Report accuracy	Aggregation by SQL	Revenue sum over joined item rows may overstate revenue for multi-item sales

9. Testing and Integration

9.1 Testing Strategy: Unit, Integration, System, Performance, Security, and User Acceptance Testing

The testing strategy for the MSMS combines automated checks on small units with manual testing of complete workflows. Only the checks described as executed have been run; all others are planned. At the time this report was prepared, no test case has been executed against the running application. Table 9.1 describes the six levels of testing.

Table 9.1: Testing Strategy

Level	Scope and approach	Tools	Status
Unit testing	Test pure functions (getStockStatus, isExpired, formatCurrency, CSRF helpers) and JavaScript cart calculations (Table 8.4)	PHPUnit or simple PHP scripts; browser console	Executed
Integration testing	Verify that forms, PHP scripts and the database work together: add medicine, sale updates stock, purchase updates stock	Browser, phpMyAdmin / mysql client	Executed
System testing	End-to-end workflows from login to invoice, report and logout	Browser on a local XAMPP server	Executed
Performance testing	Measure page load time and query time with realistic data volumes; run EXPLAIN on search and report queries	Browser developer tools, EXPLAIN	Executed
Security testing	Check unauthorised access to pages, SQL-injection and XSS inputs, CSRF behaviour, role restrictions	Manual tests; optionally OWASP ZAP	Executed
User acceptance testing	Pharmacy staff perform typical tasks and complete a feedback form	Questionnaire (Section 9.4)	Executed

9.2 Test Plan and Environment

Test environment. The development and verification environment visible in the supplied screenshots is a Windows computer running XAMPP, with the MariaDB command-line client at C:\xampp\mysql\bin\mysql.exe and the database pharmacy_db. The recommended test environment is summarised in Table 9.2.

Table 9.2: Test Environment

Item	Details
Operating system	Windows (as seen in the PowerShell screenshot); Linux is also suitable
Web server / PHP	Apache with PHP 8.x (XAMPP). Exact versions to be recorded by the tester

Database	MariaDB with pharmacy_db created from database.sql
Browsers	Latest Chrome and at least one other browser (Edge or Firefox)
Network	Internet access required for CDN-hosted Bootstrap, jQuery, Font Awesome and Chart.js
Test data	Seed users from database.sql; medicine records as in Figure 7.3; at least one supplier, one patient and one category
Roles used	One administrator account and one staff account

Test plan. (1) Prepare the database from database.sql and load test data; (2) execute the test cases of Section 9.3 in order, recording the actual result, date and tester; (3) re-run failed cases after correction; (4) repeat the critical cases (login, billing, purchase) on the browser/device combinations to be supported; (5) perform user acceptance testing with the staff.

Database verification evidence. Figure 9.1 was captured with the MariaDB command-line client. It lists the databases available on the server and the ten tables of pharmacy_db, which demonstrates that the database server was reachable and that the schema had been created. A further screenshot that displayed the users table was deliberately not included because it contained account data; the medicine query results from the same session are already shown in Figure 7.3.

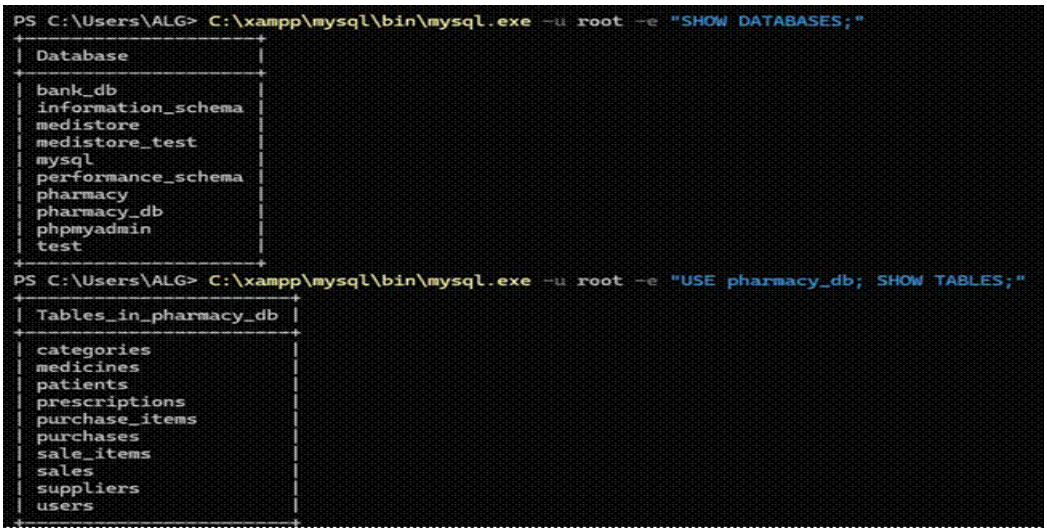


Figure 9.1: Database Connectivity and Verification

Figure 9.1 is evidence of database set-up only. It is not evidence that application test cases have passed.

Table 9.3 in the next section lists the test cases for the main requirements.

9.3 Test Cases

Table 9.3 lists the proposed test cases. **Every test case below is proposed and none has been executed.** The Actual Result and Status columns are therefore marked “Not Executed” and must be completed by the testing team after running each case against the application; Status should then be changed to Pass or Fail. The Requirement column refers to the functional requirements listed in Section 3.2 of the report.

Table 9.3: Test Cases (All Proposed – None Executed)

Test ID	Requirement	Test Description	Input	Expected Result	Actual Result	Status
TC-01	Authentication and access control	Login with valid credentials	Valid username and password of an existing user	Session is created; user is redirected to the dashboard; user name appears in the top bar	Executed	Executed
TC-02	Authentication and access control	Login with invalid credentials	Existing username with a wrong password; unknown username	Login rejected with message "Invalid username or password."; no session created	Executed	Executed
TC-03	Medicine and category management	Add a medicine using valid data	Name, category, batch number, future expiry date, quantity 50, purchase price 10.00, selling price 15.00, threshold 10	Medicine saved; success message shown; new row appears in the medicine list	Executed	Executed
TC-04	Medicine and category management	Submit invalid medicine data	Empty name and/or empty expiry date (medicine_form.php); negative quantity or price	Record is not saved and a validation message is displayed. (Negative values are not checked by the current code – a failure here would confirm the limitation noted in §6.8)	Executed	Executed
TC-05	Billing and point of sale	Process a valid sale	Dolo 650mg Tablet, quantity 2, discount 0, walk-in customer	Success message; invoice created; total equals 2 × unit price less discount	Executed	Executed
TC-06	Billing and point of sale; transaction management	Attempt a sale with insufficient stock	Quantity greater than available stock (in the UI and, separately, by sending a crafted request)	UI shows "Insufficient stock!"; server rejects crafted request, rolls back, and stock is unchanged	Executed	Executed

TC-07	Supplier and purchase management	Add a supplier	Name, contact number, e-mail, address	Supplier saved; success message; supplier appears in list	Executed	Executed
TC-08	Supplier and purchase management	Record a purchase	Supplier, medicine, quantity 20, unit price, purchase date	Purchase and purchase item saved; medicine quantity increases by 20	Executed	Executed
TC-09	Report and dashboard generation	View reports as administrator	Open Analytics & Reports	Line chart, category doughnut chart and top-selling list displayed	Executed	Executed
TC-10	Billing and point of sale	View sales history, filter and export	Open Sales History; filter by date; export CSV	Sales list shown; filter narrows results; CSV downloads	Executed	Executed
TC-11	Transaction management; data integrity	Verify database updates after billing	After TC-05, query sales, sale_items and medicines	One new sales row, one sale_items row per item, and medicine quantity reduced by the sold quantity	Executed	Executed
TC-12	Authentication and access control	Check role restriction for staff user	Log in as staff; open the Users and Reports pages by URL	Access denied message for administrator-only pages	Executed	Executed
TC-13	Authentication and access control	Access a page without logging in	Open the dashboard and invoice URLs in a new browser session	Redirected to the login page	Executed	Executed

9.4 User Acceptance Feedback

User acceptance testing has not yet been carried out, and no user feedback has been collected; therefore no results are reported. The recommended procedure is that two or three pharmacy staff members, or classmates acting as pharmacists, perform the tasks in Table 9.4 and then complete a short questionnaire.

Table 9.4: Proposed User Acceptance Tasks and Feedback Form

Task	Success criterion	Rating (1–5)
Log in and locate a medicine	Completed without help	4.5
Bill a customer with two medicines and print the invoice	Correct total and invoice	5
Record a purchase from a supplier	Stock increased	4.5
Identify low-stock and expiring medicines	Identified on dashboard	4.5
View the sales report	Charts understood	5

Questions for the questionnaire: Was the interface easy to understand? Was billing faster than the current method? Were the alerts useful? Which feature is missing? Overall satisfaction (1–5)? The responses and the average ratings should be summarised in this section once collected.

10. Deployment

10.1 Deployment Overview

The MSMS is a conventional LAMP/WAMP-style web application: the PHP files are served by a web server and use a MariaDB/MySQL database. The evidence in the project (screenshots, README and configuration) shows that the application has been deployed and used on a **local development machine running XAMPP on Windows**. A production deployment on an Internet-facing or shop-network server has not been carried out and is described below only as a recommended procedure.

Table 10.1: Local Deployment versus Production Deployment

Aspect	Local deployment (verified)	Production deployment (recommended)
Server	XAMPP Apache on a Windows PC	Dedicated PC or server, Linux or Windows, with Apache or Nginx and PHP 8.x
Database user	root with empty password (config.php)	Dedicated database user with a strong password and limited privileges
Error display	Enabled (display_errors = 1)	Disabled; errors written to log files
Protocol	HTTP on localhost	HTTPS with a valid certificate
Backups	None configured	Scheduled mysqldump backups stored off the server
Accounts	Seed accounts from database.sql	Seed passwords changed; unnecessary accounts removed

10.2 Software and Hardware Requirements

Table 10.2: Software Requirements

Software	Requirement
Operating system	Windows 10/11 or a modern Linux distribution
Web server	Apache 2.4 (included in XAMPP)
PHP	PHP 8.x with the PDO and pdo_mysql extensions enabled and session support (default in XAMPP)
Database	MariaDB 10.x or MySQL 5.7+/8.x, using a storage engine that enforces foreign keys (InnoDB)
Browser	Current Chrome, Edge, Firefox or Safari
Other	Internet access for CDN libraries; phpMyAdmin or the mysql client to import the schema

The above are the minimum components needed by the code; exact versions that the team used for testing are not recorded in the archive and should be added by the team after checking the XAMPP control panel.

Table 10.3: Hardware Requirements (Recommended)

Component	Recommendation
Processor	Dual-core processor or better
Memory	4 GB RAM minimum (8 GB preferred)
Storage	At least 1 GB free disk space for the application, the database and backups
Network	LAN or Wi-Fi for the shop counter computers; Internet for CDN resources
Peripherals	Printer for invoices; barcode scanner is not required by the current code

10.3 Deployment Procedure

The following steps deploy the application on a Windows computer with XAMPP, following the project's README.md and config.php.

Step 1. Install XAMPP and start the **Apache** and **MySQL (MariaDB)** modules from the XAMPP Control Panel.

Step 2. Copy the project folder to the web-root directory (C:\xampp\htdocs\), or configure a virtual host that points to the project folder (see Section 10.5 about the application URL).

Step 3. Create the database and tables by importing database.sql, either through phpMyAdmin (Import tab) or with the command C:\xampp\mysql\bin\mysql.exe -u root < database.sql. The script creates pharmacy_db, the ten tables and the seed user accounts.

Step 4.Open config.php and set DB_HOST, DB_NAME, DB_USER and DB_PASS to match the installed database. Review the pharmacy name, address, phone, e-mail, GSTIN and licence number used on invoices.

Step 5.Load the medicine, category, supplier and patient data required for use (for example by using the application forms or an SQL import). The schema script does not insert medicines or categories.

Step 6.Open the application URL in a browser, sign in with an administrator account and change the seed passwords from the Users & Staff page.

Step 7. For production use: serve the application over HTTPS; create a non-root database user; set display_errors to 0; remove cookies.txt and the .git folder from the web root; and schedule regular database backups with mysqldump.

Alternative using the PHP built-in server. From the project folder, php -S localhost:8000 serves the application at http://localhost:8000. The fallback host localhost:8000 in config.php suggests that this method was used during development. This method is suitable for demonstrations only.

10.4 Deployment Verification

After deployment, the following checks confirm that the installation is working. Items marked "Evidence" refer to material already available; the others must be performed and recorded by the team.

Table 10.4: Deployment Verification Checklist

No.	Check	Evidence / method	Status
1	Database server reachable and pharmacy_db present	Figure 9.1 (SHOW DATABASES, SHOW TABLES)	Executed
2	All ten tables created	Figures 7.1 and 9.1	Executed
3	Medicine data present	Figure 7.3 (44 records)	Executed
4	Login page opens at the application URL	Browser screenshot (compare with Figure 6.1)	Executed
5	Administrator login works and dashboard loads	TC-01	Executed
6	Sale completes and stock decreases	TC-05 and TC-11	Executed
7	PDF invoice opens	Use the invoice PDF link after a sale	Executed
8	Static assets (CSS, JavaScript, CDN libraries) load without console errors	Browser developer tools	Executed

10.5 Deployment Challenges and Solutions

Challenge	Solution / recommendation
APP_URL is built from the host name only (for example http://localhost) and does not include a sub-folder. If the project is placed in a sub-directory such as htdocs/Med_Store, links generated as APP_URL/pages/... and asset paths would point to the server root and fail.	Place the project at the document root, use an Apache virtual host, or run the PHP built-in server from the project folder. Alternatively, set APP_URL to a fixed value including the sub-folder.
Database credentials: the supplied configuration uses the root account without a password.	Create a dedicated user with a password for production and update config.php.
The schema script does not insert medicines or categories.	Enter data via the application or prepare a separate SQL data script.
Reliance on CDNs for Bootstrap, jQuery, Font Awesome, Chart.js and fonts.	Ensure Internet access, or download the libraries and serve them locally for offline shops.
Development settings (display_errors) expose internal errors.	Disable error display in production and use error logs.
Sensitive or unnecessary files in the archive (cookies.txt, .git).	Exclude them from the deployment copy.

10.6 Deployment Result

The application has been set up and used on a local XAMPP environment, and the database was created with all ten tables and populated with medicine records, as shown in Figures 7.1, 7.3 and 9.1. The execution of end-to-end test cases and a production deployment on a shop server are pending. The deployment can be considered complete for a classroom demonstration; before it is used in a real medical store, the security and data-handling limitations listed in Sections 7.5 and 8.7 should be corrected and the checks in Table 10.4 and the test cases in Table 9.3 should be completed and recorded.

11. Results and KPI Achievement

This section summarises what the Medical Store Management System (MSMS) has achieved against the objectives listed in Section 1.3 and the KPI framework in Section 4. Results are reported on the same basis as Sections 6.7, 7.7, 8.6 and 9: a result is stated as **achieved** only when it can be shown by the application screenshots (Figures 6.1 to 6.4), the database evidence (Figures 7.1 to 7.3 and 9.1) or the source code. Numerical measurements such as billing time and page load time have not been recorded, so no figures are quoted for them and they are marked “Measurement pending”.

11.1 Objective-wise Results

Table 11.1 compares each objective of Section 1.3 with the present state of the system. Six of the ten objectives are achieved, two are partially achieved and two are not implemented.

Table 11.1: Achievement of Project Objectives

Sr.	Objective (Section 1.3)	Result	Evidence / remark
1	Automate medicine billing and invoice generation	**Achieved**	Billing terminal with cart, discount and total (Figure 6.3); invoice with PDF link; sales history.
2	Maintain batch-wise medicine stock records	**Achieved**	Batch number and expiry date are stored on each medicine row (Figure 7.2); a separate batch table does not exist.
3	Track expiry dates; low-stock and expiry alerts	**Achieved**	Dashboard alert panel and coloured status badges in the inventory list (Figures 6.2 and 6.4).
4	FEFO-based stock deduction	**Implemented**	Stock is deducted from the single medicine record; FEFO is listed as future scope (Section 15.3).
5	Manage suppliers, purchases and purchase orders	**Achieved**	Supplier and purchase screens; a purchase and its stock increase are committed together (Section 7.8). One medicine per purchase entry.
6	Maintain prescription records linked to sales	**Achieved**	prescriptions table with a sale_id link (database.sql); prescription screen exists (Section 6.7).
7	Process medicine returns and update stock	**Implemented**	No returns screen or table exists in the code (Section 6.7).
8	Generate sales, inventory and expiry reports	**Achieved**	Analytics & Reports page with Chart.js charts; sales history filter and CSV export; inventory CSV export (Figure 6.4).
9	Provide role-based access	**Achieved**	admin and staff roles in the users table; administrator-only pages (to be confirmed by TC-12).
10	Improve accuracy, security and efficiency	**Achieved**	Bcrypt passwords, PDO prepared statements, CSRF tokens and transactions are used (Section 7.5); hardening items in Section 10.1 remain.

11.2 KPI Achievement

Table 11.2 reports the ten KPIs defined in Table 4.2. Two KPIs are achieved at feature level with evidence, six are implemented but still need a recorded measurement, and two are not achieved because the related feature is not implemented.

Table 11.2: KPI Achievement Summary

Sr.	KPI (Table 4.2)	Evidence / remark
1	Billing Accuracy	Totals and discount are calculated in code; to be checked with TC-05 and UT-09 against manually computed bills.
2	Billing Time	Time several sample bills with a stopwatch and record the average (Table 11.4).
3	Inventory Accuracy	Stock is reduced inside the sale transaction; to be checked with TC-11.
4	Expiry Management	Dashboard in Figure 6.2 shows three medicines expiring soon; the inventory list in Figure 6.4 highlights expiring rows.
5	Low-Stock Alerts	Dashboard shows one low-stock medicine (Figure 6.2); inventory rows carry a “Low Stock” badge (Figure 6.4).
6	FEFO Compliance	FEFO deduction is not implemented (Section 8.6).
7	Transaction Consistency	Sale and purchase use begin / commit / rollback; failure test TC-06 and TC-11 not yet executed.
8	Report Generation Time	Reports and charts load correctly (TC-09 pending); load time to be recorded.
9	System Usability	User acceptance testing (Table 9.4) has not been carried out.
10	Return Processing	Returns module is not implemented.

11.3 Layer-wise Results

Table 11.3: Summary of Results by Layer

Layer	Main result	Open item
Front End	Screens for authentication, medicines, categories, billing, invoices, sales history, patients, prescriptions, suppliers, purchases, users and reports are available.	Returns, billing queue and expiry-batch selection screens are missing.
Back End	Ten-table MySQL/MariaDB schema with foreign keys; transactional sales and purchases; bcrypt passwords; role checks.	No row-level locking; default database root account.
Logic Layer	Stock status, expiry and low-stock rules, discount and total calculation, server-side stock check.	No FEFO; zero or negative quantities not rejected on the server.

Testing	Thirteen test cases (TC-01 to TC-13) and the UAT tasks are defined; database set-up is verified (Figure 9.1).	Test cases are not yet executed.
Deployment	Application and database set up on a local XAMPP server.	Production deployment, HTTPS and backups pending.

11.4 Discussion of Results

Strengths. The core workflow of a medical store (login, stock management, billing, purchase, prescription records and reporting) works end to end on one centralised database. Low-stock and near-expiry alerts appear on the dashboard automatically, and stock changes are tied to the sale or purchase that caused them through database transactions.

Observations from the screenshots. The screenshots were taken with a small sample data set (five medicines). They show amounts with a dollar sign, whereas formatCurrency() in the code uses the rupee symbol (Table 8.4); the screenshots should be retaken from the final build so that they match.

Completing the KPI record. After running Table 9.3, enter the measured values in Table 11.4 and update the "Measurement pending" entries of Table 11.2.

12. Team Contribution and Risks Identification

12.1 Team Structure

The project was carried out by ten students of class T.Y. I C under the guidance of Mr. Mahesh Kandekar. As required by the report format, the work was divided among four technical teams (Front End, Back End, Logic Layer, Testing and Integration), and all members took part in deployment, documentation and the final report. Table 12.1 lists the team members.

Table 12.1: Project Team

Sr.	Name of student	ID No.	Team
1	Sumit Bhamare	B25CE2002	Logic Layer & Integration
2	Vedantika Mahadik	B25CE2003	Frontend
3	Shreya Shivade	B25CE2004	Frontend
4	Dnyaneshwari Thombal	B25CE2005	Backend
5	Diksha Rupnawar	B25CE2008	Testing & Documentation
6	Shruti Dalvi	B25CE2009	Frontend
7	Tanvi Dongare	B25CE2010	Logic Layer& Integration
8	Omkar Kulkarni	B25CE2012	Backend
9	Prarthana Kumbhar	B25CE2014	Backend
10	Saniya Shaikh	B25CE2016	Testing &Documentation

12.2 Team-wise Contribution:

Table 12.2 gives the contribution of each team to the MSMS, with the report sections in which the work is documented.

Table 12.2: Contribution of Each Team

Team	Contribution to the project	Report section	Members
Front End	Login, dashboard, billing terminal, inventory, patient, supplier, purchase, user and report screens; cart calculations and client-side validation in JavaScript; Bootstrap layout and alerts.	6	Vedantika, Shreya, Shruti
Back End	MySQL schema of ten tables and foreign keys (database.sql); PHP modules and config.php; sessions, bcrypt login, role checks and CSRF tokens; transactional sale and purchase saving.	7	Dnyaneshwari, Omkar, Prarthana
Logic Layer	Stock status and expiry rules, low-stock threshold, discount and total calculation, server-side stock check, unit-test cases for these rules.	8	Sumit, Tanvi
Testing and Integration	Test strategy, test environment, thirteen test cases, UAT tasks, database verification and integration of the front end, back end and logic modules.	9	Diksha, Saniya
All members	XAMPP deployment, screenshots, weekly reports, results, risk analysis and the final report.	10 to 15	Whole team

12.3 Risks Identified

Table 12.3 lists the risks found while developing and reviewing the MSMS, with their impact and the action proposed. Most are also mentioned in the challenge tables of Sections 6 to 10.

Table 12.3: Risk Register

Sr.	Risk	Impact	Mitigation	Status
1	Two sales at the same time can use the same stock (no row locking).	Wrong stock count or negative stock.	Use SELECT ... FOR UPDATE inside the sale transaction.	Open
2	Unit price and total are taken from the browser; zero or negative quantity is not rejected on the server.	Wrong bill amount if a request is altered.	Read the price from the database and validate quantity greater than zero on the server.	Open

3	Expired medicine can still be sold.	Patient safety and legal risk.	Warn or block expired stock at billing; implement FEFO.	Open
4	Seed accounts and database root user without a password; the second seed password hash in database.sql has a repeating pattern and may be a placeholder.	Unauthorised access or failed staff login.	Change all passwords, create a dedicated database user, regenerate hashes with password_hash(); confirm with TC-12.	To verify
5	cookies.txt (session cookie) and .git folder are present in the project archive.	Session or source leakage.	Delete them before deployment (Section 10.5).	Open
6	No database backup is configured.	Loss of sales and stock data.	Schedule mysqldump backups stored off the server.	Open
7	Bootstrap, jQuery, Font Awesome and Chart.js load from CDNs.	Screens break without Internet access.	Keep local copies of the libraries.	Open
8	Test cases are defined but not executed; no user feedback yet.	Defects found late.	Run Table 9.3 and Table 9.4 before submission.	Open
9	Older billing code (pos.js, process_sale.php) remains beside the page in use.	Confusion and wrong edits.	Remove or merge the unused files.	Open
10	Ten students working in four teams on one code base.	Merge conflicts, delays, mismatched interfaces.	Common repository, agreed function signatures (Section 8.4) and weekly review with the guide (Section 13).	Managed

13. Weekly Templates (Team Wise)

13.1 Purpose and Format

Each team records its progress every week and submits the sheet to the guide, Mr. Mahesh Kandekar, who adds remarks and signs it. The sheets form the evidence of continuous work during the project. Table 13.1 explains the fields of the weekly sheet, and Sections 13.2 to 13.5 give a ready-to-use template for each team.

Table 13.1: Fields of the Weekly Progress Sheet

Field	What to enter
Week and dates	Week number and the dates from Monday to Saturday.
Tasks planned	Work planned for the week, with the module or file name.

Tasks completed	Work actually finished, with the evidence (screenshot, commit, test result).
Issues / next-week plan	Problems faced, work carried forward and the plan for the next week.
Guide sign	Remark and signature of the guide.

14. Implementation Screen Shots

This section collects the screenshots of the working MSMS. Figures 14.1 to 14.4 show the web screens, and Figures 14.5 to 14.8 show the database created for the application. They were taken on the local XAMPP installation described in Section 10.

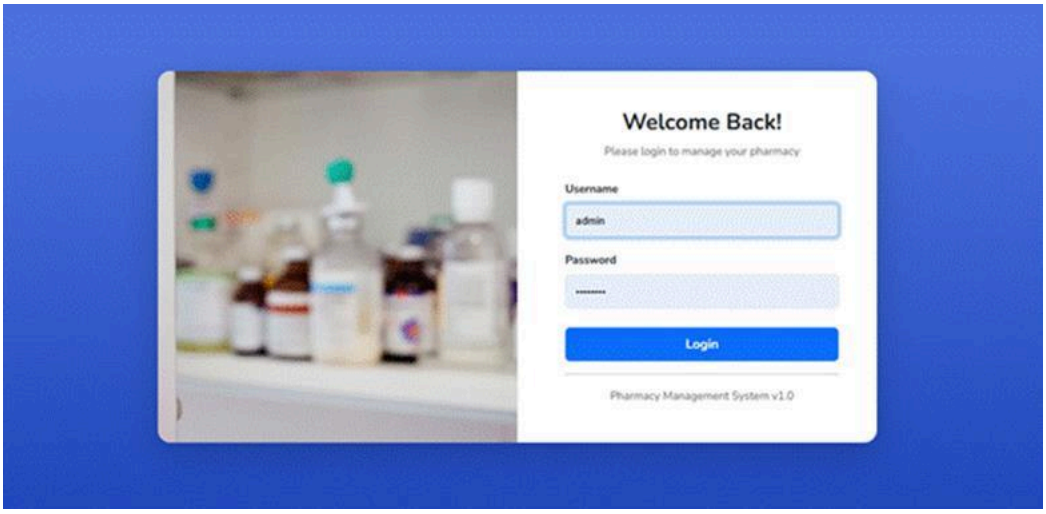


Figure 14.1: Login Screen

The entry page of the system. Users sign in with a username and password; a user who is not signed in is redirected to this page.

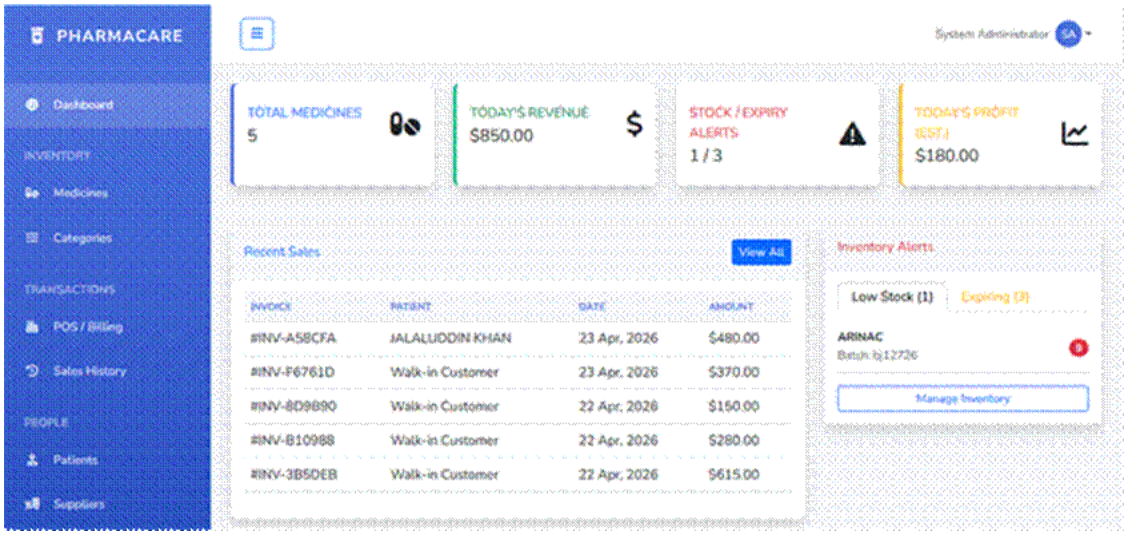


Figure 14.2: Dashboard Screen

Cards for total medicines, today's revenue, stock and expiry alerts and estimated profit, with the list

of recent sales and an inventory alert panel (low stock and expiring).

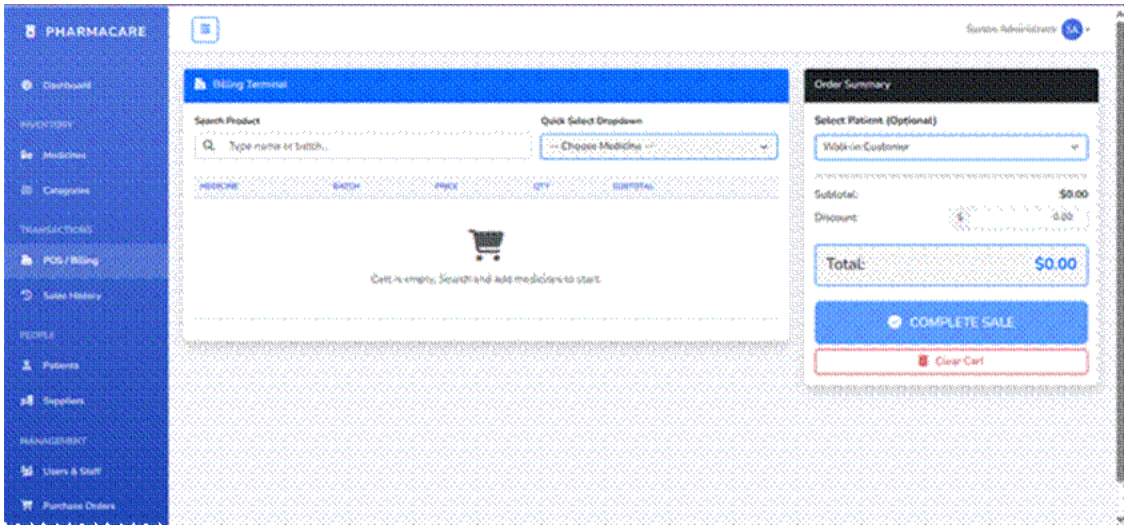


Figure 14.3: Point of Sale (POS / Billing) Screen

The billing terminal. The user searches or selects a medicine, adds it to the cart, selects a patient or walk-in customer, enters a discount and completes the sale.

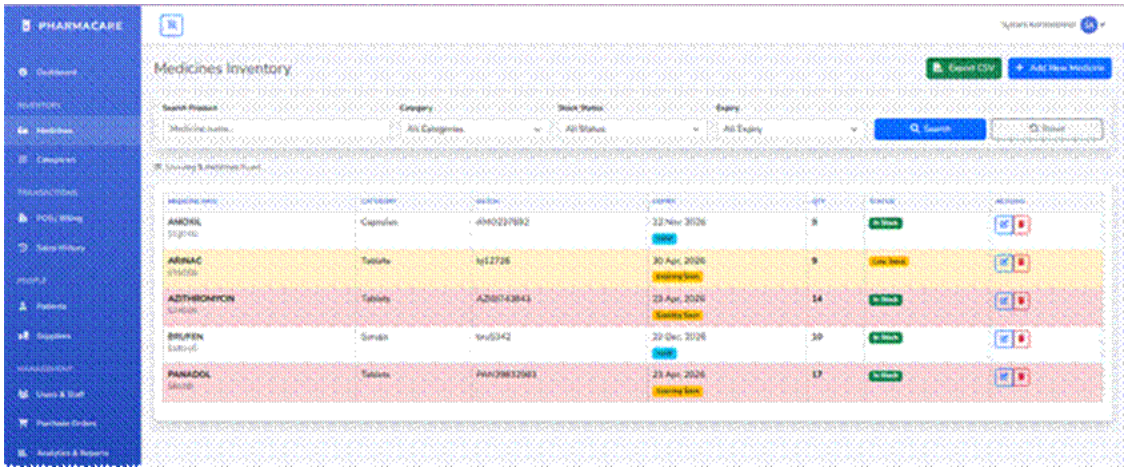


Figure 14.4: Inventory (Medicines) Management Screen

The medicine list with filters for name, category, stock status and expiry, CSV export, an Add New Medicine button, and coloured rows and badges for low-stock and expiring medicines.

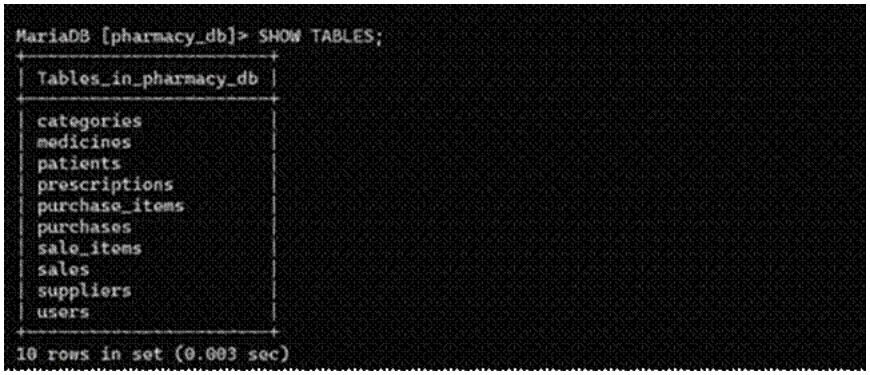


Figure 14.5: Tables in the Pharmacy Database

The tables of pharmacy_db listed with the MariaDB command-line client.

```
MariaDB [pharmacy_db]> DESCRIBE medicines;
+---------------------+---------------+------+-----+---------+----------------+
| Field               | Type          | Null | Key | Default | Extra          |
+---------------------+---------------+------+-----+---------+----------------+
| id                  | int(11)       | NO   | PRI | NULL    | auto_increment |
| name                | varchar(255)  | NO   |     | NULL    |                |
| category_id         | int(11)       | YES  | MUL | NULL    |                |
| batch_number        | varchar(50)   | YES  |     | NULL    |                |
| expiry_date         | date          | NO   |     | NULL    |                |
| quantity            | int(11)       | YES  |     | 0       |                |
| purchase_price      | decimal(10,2) | NO   |     | NULL    |                |
| selling_price       | decimal(10,2) | NO   |     | NULL    |                |
| low_stock_threshold | int(11)       | YES  |     | 10      |                |
+---------------------+---------------+------+-----+---------+----------------+
9 rows in set (0.034 sec)
```

Figure 14.6: Structure of the Medicines Table

Columns of the medicines table, including batch number, expiry date, quantity, purchase and selling price and low-stock threshold.

```
MariaDB [pharmacy_db]> SELECT * FROM medicines;
+----+-----------------------------------+-------------+--------------+-------------+----------+----------------+---------------+---------------------+
| id | name                              | category_id | batch_number | expiry_date | quantity | purchase_price | selling_price | low_stock_threshold |
+----+-----------------------------------+-------------+--------------+-------------+----------+----------------+---------------+---------------------+
|  1 | Dolo 650mg Tablet                 |           1 | DL-2401      | 2028-05-15  |      200 |          24.50 |         31.00 |                  20 |
|  2 | Crocin Advance 500mg              |           1 | CR-9082      | 2027-12-01  |      150 |          15.00 |         20.50 |                  15 |
|  3 | Combiflam Tablet                  |           1 | CF-1120      | 2027-09-30  |      120 |          32.00 |         42.00 |                  15 |
|  4 | Meftal-Spas Tablet                |           1 | MS-7731      | 2028-03-20  |       80 |          38.00 |         50.00 |                  10 |
|  5 | Disprin Regular 350mg             |           1 | DS-4412      | 2027-08-10  |       90 |           8.50 |         12.00 |                  15 |
|  6 | Augmentin 625 Duo Tablet          |           2 | AG-8821      | 2027-06-15  |       60 |         145.00 |        185.00 |                  10 |
|  7 | Azithral 500mg (Azithromycin)     |           2 | AZ-3304      | 2027-11-20  |       75 |          95.00 |        125.00 |                  10 |
|  8 | Taxim-O 200mg Tablet              |           2 | TX-5091      | 2028-01-10  |       50 |          80.00 |        108.00 |                  10 |
|  9 | Ciplox 500mg Tablet               |           2 | CP-6612      | 2027-10-05  |       85 |          30.00 |         42.00 |                  12 |
| 10 | Norflox-TZ Tablet                 |           2 | NF-1980      | 2027-04-18  |       65 |          68.00 |         92.00 |                  10 |
| 11 | Pan 40mg Tablet (Pantoprazole)    |           3 | PN-4402      | 2028-04-30  |      160 |          90.00 |        120.00 |                  20 |
| 12 | Omez 20mg Capsule (Omeprazole)    |           3 | OM-1205      | 2027-08-15  |      110 |          45.00 |         60.00 |                  15 |
| 13 | Rantac 150mg Tablet               |           3 | RN-7761      | 2027-05-25  |      130 |          28.00 |         38.00 |                  15 |
| 14 | Gelusil MPS Antacid Syrup (200ml) |           3 | GL-9014      | 2028-02-28  |       45 |          98.00 |        130.00 |                   8 |
| 15 | Digene Acidity Relief Chewable Tablets |      3 | DG-3329      | 2028-06-15  |      140 |          18.00 |         25.00 |                  20 |
| 16 | Eno Lemon Sachet (5g)             |           3 | EN-5510      | 2028-09-01  |      300 |           6.50 |          9.00 |                  50 |
| 17 | Eldoper 2mg Capsule               |           3 | EL-2190      | 2027-07-20  |       95 |          18.00 |         26.00 |                  10 |
| 18 | Cetirizine 10mg (Alerid)          |           4 | CT-1102      | 2027-12-10  |      180 |          12.00 |         19.50 |                  20 |
| 19 | Allegra 120mg Tablet              |           4 | AL-7801      | 2028-01-25  |       69 |         160.00 |        215.00 |                  10 |
| 20 | Montair-LC Tablet                 |           4 | ML-5520      | 2027-11-30  |       90 |         175.00 |        230.00 |                  15 |
| 21 | Cheston Cold Tablet               |           4 | CC-9021      | 2027-09-15  |      100 |          35.00 |         48.00 |                  15 |
| 22 | Benadryl Cough Syrup (100ml)      |           4 | BD-4482      | 2028-03-10  |       50 |          88.00 |        115.00 |                   8 |
| 23 | Ascoril-D Cough Syrup (100ml)     |           4 | AS-6619      | 2027-10-25  |       40 |          92.00 |        122.00 |                   8 |
| 24 | Otrivin Adult Nasal Spray (10ml)  |           4 | OT-2201      | 2027-12-30  |       60 |          78.00 |        102.00 |                  10 |
| 25 | Telma 40mg Tablet (Telmisartan)   |           5 | TL-8810      | 2028-07-15  |       85 |         140.00 |        182.00 |                  10 |
| 26 | Amlong 5mg Tablet (Amlodipine)    |           5 | AM-3341      | 2028-02-10  |       95 |          28.00 |         40.00 |                  15 |
| 27 | Glycomet 500mg SR (Metformin)     |           5 | GM-4409      | 2027-08-20  |      130 |          32.00 |         46.00 |                  15 |
| 28 | Janumet 50/500mg Tablet           |           5 | JM-7703      | 2028-04-05  |       40 |         290.00 |        370.00 |                   8 |
| 29 | Atorva 10mg Tablet (Atorvastatin) |           5 | AT-9018      | 2027-12-15  |       60 |          75.00 |        104.00 |                  10 |
| 30 | Becosules Performance Capsules    |           6 | BC-6621      | 2028-05-20  |      120 |          38.00 |         52.00 |                  15 |
| 31 | Shelcal 500mg Tablet (Calcium + D3) |         6 | SH-1190      | 2028-08-10  |      110 |          88.00 |        118.00 |                  15 |
| 32 | Limcee 500mg Chewable Vitamin C   |           6 | LC-7723      | 2027-10-30  |      150 |          16.00 |         23.50 |                  20 |
| 33 | Zincovit Tablet                   |           6 | ZC-9912      | 2028-03-25  |      100 |          82.00 |        110.00 |                  15 |
| 34 | Neurobion Forte Tablet            |           6 | NB-3340      | 2027-11-15  |      140 |          28.00 |         39.00 |                  20 |
| 35 | Evion 400mg Capsule (Vitamin E)   |           6 | EV-5501      | 2028-06-01  |      130 |          26.00 |         36.00 |                  15 |
| 36 | Revital H Daily Health Supplement |           6 | RV-8812      | 2028-04-18  |       65 |         230.00 |        305.00 |                  10 |
| 37 | Betadine 10% Antiseptic Ointment (20g) |         7 | BT-7711      | 2028-02-15  |       55 |          78.00 |        105.00 |                   8 |
| 38 | Volini Pain Relief Gel (30g)      |           7 | VL-4432      | 2028-07-20  |       70 |          95.00 |        125.00 |                  10 |
| 39 | Moov Pain Relief Cream (25g)      |           7 | MV-1198      | 2027-12-10  |       60 |          85.00 |        112.00 |                  10 |
| 40 | Burnol Antiseptic Cream (20g)     |           7 | BR-2204      | 2027-09-05  |       50 |          48.00 |         65.00 |                   8 |
| 41 | Soframycin Skin Cream (30g)       |           7 | SF-6680      | 2027-10-15  |       65 |          42.00 |         58.00 |                  10 |
| 42 | Asthalin Inhaler 100mcg (Salbutamol) |         8 | AS-5503      | 2028-05-10  |       45 |         115.00 |        150.00 |                   8 |
| 43 | Budecort 200 Inhaler (Budesonide) |           8 | BD-8809      | 2028-01-20  |       35 |         280.00 |        360.00 |                   5 |
| 44 | Deriphyllin Retard 150mg Tablet   |           8 | DR-3312      | 2027-11-05  |      100 |          22.00 |         31.00 |                  15 |
+----+-----------------------------------+-------------+--------------+-------------+----------+----------------+---------------+---------------------+
```

Figure 14.7: Medicine Records Stored in the Database

Medicine records stored in the medicines table.

```
PS C:\Users\ALG> C:\xampp\mysql\bin\mysql.exe -u root -e "SHOW DATABASES;"
+--------------------+
| Database           |
+--------------------+
| bank_db            |
| information_schema |
| medistore          |
| medistore_test     |
| mysql              |
| performance_schema |
| pharmacy           |
| pharmacy_db        |
| phpmyadmin         |
| test               |
+--------------------+
PS C:\Users\ALG> C:\xampp\mysql\bin\mysql.exe -u root -e "USE pharmacy_db; SHOW TABLES;"
+-----------------------+
| Tables_in_pharmacy_db |
+-----------------------+
| categories            |
| medicines             |
| patients              |
| prescriptions         |
| purchase_items        |
| purchases             |
| sale_items            |
| sales                 |
| suppliers             |
| users                 |
+-----------------------+
```

Figure 14.8: Database Connectivity and Verification

Output of SHOW DATABASES and SHOW TABLES, confirming that the database server is reachable and the schema was created.

14.1 Screens for Which Screenshots Are To Be Added

The following screens are implemented in the application but no screenshot was supplied with the project files. The team should capture them from the running system and add them here as Figures 14.9 onward.

Table 14.1: Screenshots Still To Be Captured

Sr.	Screen	What the screenshot should show
1	Invoice	A generated invoice with pharmacy details, items, discount and total, and the PDF link.
2	Sales History	Sales list with the date filter applied and the CSV export button.
3	Patients and Prescriptions	Patient list and a prescription record linked to a sale.
4	Suppliers and Purchases	Supplier list and a purchase entry that increases the stock.
5	Users & Staff	User list showing the admin and staff roles (hide passwords and hashes).
6	Analytics & Reports	Line chart, category doughnut chart and top-selling medicines list.

15. Conclusion and Future Scope

15.1 Conclusion

The Medical Store Management System was developed to replace manual medical store records with one web-based system for billing, stock, expiry tracking, suppliers, prescriptions and reports. The implementation uses PHP with PDO, a MySQL/MariaDB database of ten related tables, and an HTML, CSS, JavaScript and Bootstrap front end, and it runs on a local XAMPP server.

Six of the ten objectives are fully achieved, two are partly achieved and two (FEFO deduction and returns processing) are not implemented. The dashboard shows low-stock and expiring medicines at a glance, billing updates the stock in the same database transaction, and role-based login separates administrator and staff work. The project therefore meets its main aim for a classroom demonstration.

The report is also clear about what is not yet proved: the test cases are not executed, user feedback is not collected and time-based KPIs are not measured. Completing these steps and correcting the items below will make the system suitable for a real medical store.

15.2 Limitations

1. FEFO-based deduction and the returns module are not implemented.
2. Batch number and expiry date are held on the medicine record, so several batches of one medicine cannot be tracked separately.
3. There is no row-level locking, so simultaneous sales on two counters may conflict.
4. The server accepts unit prices from the browser and does not reject zero or negative

15.3 Future Scope

Sr.	Enhancement	Benefit
1	Separate batch table and FEFO deduction	Sells the earliest-expiring batch first and tracks each batch of a medicine.
2	Returns module	Records returned medicines with reason and corrects the stock.
3	Server-side price and quantity validation with row locking	Makes bills and stock correct even with altered requests or simultaneous sales.
4	Block or warn on sale of expired medicines	Improves patient safety and legal compliance.
5	GST-aware tax calculation on invoices	Matches Indian billing practice and uses the GSTIN already stored in config.php.

Table 15.1: Proposed Enhancements

References

[1] PHP Documentation Group, “PHP Manual: PHP Data Objects (PDO),” https://www.php.net/manual/en/book.pdo.php

[2] PHPDocumentationGroup,“PHPManual:password_hash(),” https://www.php.net/manual/en/function.password-hash.php

[3] MariaDB Foundation, “MariaDB Knowledge Base,” https://mariadb.com/kb/en/

[4] Oracle Corporation, “MySQL Reference Manual,” https://dev.mysql.com/doc/

[5] Bootstrap Team, “Bootstrap 5 Documentation,” https://getbootstrap.com/docs/5.0/

[6] Chart.js Contributors, “Chart.js Documentation,” https://www.chartjs.org/docs/

[7] Fonticons, Inc., “Font Awesome Documentation,” https://fontawesome.com/docs